

Operating Systems – Master Notes (Unit I & Unit II)

RV University | CS1103 | Semester II | CIE 1 Preparation

Table of Contents

UNIT I – Introduction to Operating Systems

1. Definition & Role of an OS
2. Types of Operating Systems
3. Functions of an OS
4. OS Services
5. Abstract View of a Computer System
6. Computer System Organization
7. OS Structure (Monolithic, Layered)
8. Booting a Computer
9. System Calls
10. Virtualization & Virtual Machines

UNIT II – Process Management

11. What is a Process?
12. Memory Layout of a Process
13. Process Control Block (PCB)
14. Process State Models
15. Process Creation & Termination
16. Threads & Multithreading
17. Interprocess Communication (IPC)
18. Types of Schedulers
19. Scheduling Criteria
20. Scheduling Algorithms
 - 20.1 FCFS
 - 20.2 Round Robin (RR)
 - 20.3 SJF (Non-Preemptive)
 - 20.4 SRTF (Preemptive SJF)
 - 20.5 Algorithm Comparison Table
21. Key Formulae Quick Reference

UNIT I — INTRODUCTION TO OPERATING SYSTEMS

1. Definition & Role of an OS

1.1 Core Definitions

Definition 1 (User-centric): An OS is software that manages a computer's hardware, provides a basis for application programs, and acts as an intermediary between the user and the hardware.

Definition 2 (System-centric): An OS is a program that controls the execution of application programs and acts as an interface between applications and computer hardware.

Definition 3 (Kernel-centric): The OS is the one program running at all times on the computer — called the kernel. Everything else is either system programs (associated with the OS but not part of the kernel) or application programs.

Analogy: Like a government — performs no useful function by itself, but provides an environment within which other programs can do useful work.

1.2 Three Categories of Software

Category	Description	Examples
Kernel	Core of the OS; always resident in memory	Linux kernel, Windows NT kernel
System Programs	Associated with OS; not strictly part of kernel	Shells (<code>bash</code>), compilers, editors
Application Programs	All other programs; not related to OS operation	Chrome, Word, games

1.3 Dual Role of the OS

Role	Description
Resource Allocator	Manages CPU time, memory space, storage, I/O devices; resolves conflicts between competing requests
Control Program	Manages execution of user programs to prevent errors and improper use of the computer; especially concerned with I/O device operation and control

1.4 The Kernel

- Central part of the OS managing CPU time (scheduler) and memory.
- Manages CPU time by deciding which process runs on the CPU next and for how long.
- The kernel itself is a program that resides in main memory and is executed by the CPU.
- When the kernel (scheduler) executes, it takes decisions; after the stipulated time it forcibly stops the user program and regains control.
- User programs time-share the CPU, memory, and devices.

1.5 Mobile OS Architecture

Mobile OS (e.g., iOS, Android) = Core Kernel + Middleware

- **Middleware:** Set of software frameworks providing additional services to application developers (databases, multimedia, graphics).
 - **Example:** Android = Linux kernel + middleware (SQLite, OpenGL, WebKit, etc.)
-

2. Types of Operating Systems

2.1 Complete Classification

OS Type	Target Hardware	Key Characteristics	Examples
Mainframe OS	Mainframes (large data centers)	Batch, transaction, timesharing; heavy I/O orientation	OS/390, Linux variants
Server OS	Large PCs, workstations, mainframes	Serves multiple users over network; print/file/web services	Solaris, FreeBSD, Linux, Windows Server
Multiprocessor OS	Multi-CPU systems	Parallel computing; shared memory	Windows, Linux on multicore
Personal Computer OS	Desktop, laptop	Good single-user support; multiprogramming	Linux, FreeBSD, Windows 7/8/10, macOS
Handheld OS	Smartphones, tablets, PDAs	Touch-oriented; power-efficient	Android, iOS
Embedded OS	Microwave ovens, cars, DVD players	No untrusted software; ROM-based; no protection needed	Embedded Linux, QNX, VxWorks
Sensor-Node OS	Tiny battery-powered sensor nodes	Event-driven; minimal RAM; power-efficient	TinyOS, Contiki, RIOT, FreeRTOS
Real-Time OS (RTOS)	Industrial control, avionics, military	Time as key parameter; hard/soft deadlines	VxWorks, RTLinux
Smart Card OS	Credit-card-sized CPU chips	Severe power/memory constraints; single or multi-function	JavaCard, proprietary

2.2 Detailed Notes on Each Type

Mainframe Operating Systems

- **Services provided:** Three kinds — batch (routine jobs, no user), transaction processing (large numbers of small requests — bank cheque processing, airline reservations), and timesharing (multiple remote users querying a database).
- **Example:** IBM OS/390. Gradually replaced by UNIX variants like Linux.

Server Operating Systems

- Allow users to share hardware and software resources over a network.
- **Services:** print service, file service, web service.
- **Example:** Internet providers run server machines; websites use servers to store pages and handle incoming requests.

Multiprocessor Operating Systems

- Multiple CPUs connected into a single system → parallel computation.
- **Also called:** parallel computers, multi-computers, multiprocessors depending on connection type and what is shared.
- **Key challenge:** having applications make effective use of multiple cores.

Embedded Operating Systems

- All software is in ROM — no user-installed software possible.
- No protection between applications needed → simpler design.
- Must be deterministic and reliable.
- **Examples:** Parking meters, airbag controllers, microwave ovens.

Sensor-Node Operating Systems

- Sensor nodes = tiny computers with built-in radios; battery-powered.
- OS must be small, simple, and event-driven due to limited RAM and battery constraints.
- Programs are loaded in advance — no dynamic downloads.
- Must be robust against node failure as batteries die.
- **Primary example:** TinyOS — event-driven, used on Telos, MICAz nodes.
- **Others:** Contiki, RIOT, Mantis, FreeRTOS.

Real-Time Operating Systems (RTOS)

Type	Definition	Consequence of Missing Deadline	Example
Hard RTOS	Must provide absolute guarantee that an action occurs by a certain time	Catastrophic (car welding robot timing failure ruins car)	Industrial process control, avionics, missiles
Soft RTOS	Missing an occasional deadline is acceptable and causes no permanent damage	Degraded quality, not disaster	Multimedia playback, smartphones

Smart Card Operating Systems

- Smallest OS category — runs on CPU chips embedded in credit-card-sized devices.
- Powered by contacts in reader (or inductively for contactless cards).
- **Some are Java-oriented:** ROM holds a JVM interpreter; Java applets downloaded and interpreted.
- Multi-applet cards require scheduling, resource management, and protection (handled by a primitive OS).

3. Functions of an OS

Function	Description
Bootting	Starting the computer and loading the OS
Memory Management	Allocating and deallocating memory to processes
Processor Management	Scheduling which process gets CPU and for how long
Device Management	Managing I/O devices via device drivers
File Management	Creating, organizing, storing, retrieving, and protecting files

I/O Management	Providing uniform interface for diverse I/O devices
Security	Protecting system resources from unauthorized access
Control Over System Performance	Monitoring response time, resource usage
Job Accounting	Tracking CPU time, memory usage for billing/auditing
Error-Detecting Aids	Detecting and responding to hardware/software errors
Coordination	Coordinating between software and users
Basic Computer Tasks	Handling interrupts, traps, system calls

4. OS Services

4.1 Program Development

- OS provides editors, debuggers, and compilers as utility programs (application development tools).
- These are *not* strictly part of the OS kernel but are supplied with it.
- **Examples:** Visual C++ on Windows; Eclipse, Code::Blocks on Windows/Linux.

4.2 Program Execution

- **Steps required before a program executes:**
 1. Resident set (code + data + stack) loaded from disk into main memory.
 2. I/O devices and file handles initialized.
 3. Resources (memory, devices) allocated.
- OS handles all scheduling duties transparently.

Resident Set: The required contents of a program that must be in main memory for it to execute: Code + Data + Stack.

Example: When PowerPoint opens a .pptx file, main memory contains: Windows kernel + PowerPoint executable + data structures for the open PPT instance + the PPT file contents loaded from HDD.

4.3 Access to I/O Devices

- **Device Driver:** A program bundled with the OS that understands a specific device controller, exposing a uniform interface to the rest of the OS.
- **Hardware Abstraction Layer (HAL):** Abstracts access to different hardware peripherals with a common set of APIs.
- **Example:** Both a serial port and a filesystem expose the same `open()`, `read()`, `write()` interface to application programs.

4.4 Controlled Access to Files

- OS understands the structure of I/O devices (disk, tape) and file system format.
- OS reads files as a stream of bytes — the application (e.g., PowerPoint) understands the file's internal structure.
- **On multi-user systems:** OS provides protection mechanisms (e.g., read-only access) to control file access.
- **Example:** When an instructor edits a file on Moodle while students simultaneously download it — OS ensures controlled concurrent access.

4.5 System Access

- Controls access to the system as a whole and to specific system resources.
- **Authentication:** Username/password login enforces controlled access.
- **Resource protection:** Prevents unauthorized access; resolves resource contention.
- **Example:** OS prevents opening a file for update while another user is currently writing to it.

4.6 Error Detection and Response

Sources of errors:

- **Internal hardware errors:** Device failure, memory parity error.
- **External hardware errors:** Device malfunction.
- **Software errors:** Division by zero, illegal memory access (corrupted pointer), accessing a non-existent drive.
- **File access violations:** Missing file, insufficient access rights.

- **User errors:** Attempting to rename a file to a name that already exists (OS prevents this).

4.7 Accounting

- OS collects usage statistics for resources and monitors performance parameters (response time).
- **Useful for:** capacity planning (anticipating future enhancements), performance tuning, billing on multiuser systems.
- **Examples:** Disk usage monitoring, battery health, CPU utilization graphs, network bandwidth usage.

5. Abstract View of a Computer System

5.1 Layered View

USERS	← User's View
APPLICATION PROGRAMS (compilers, browsers, development kits, etc.)	
OPERATING SYSTEM	← System's View
COMPUTER HARDWARE (CPU, Memory, I/O devices)	

5.2 User's View vs System's View

Perspective	What the OS looks like
User's View	An interface that allows running applications based on what the user interacts with
System's View	A resource allocator (CPU time, memory space, I/O devices) and conflict resolver; a control program preventing errors and improper I/O device usage

5.3 How the OS Manages Resources (Key Insight)

The OS is unusual as a control mechanism in two respects:

1. The OS functions like ordinary computer software — it is a program executed by the processor.
2. The OS frequently relinquishes control — it must depend on the processor to regain control after user programs execute.

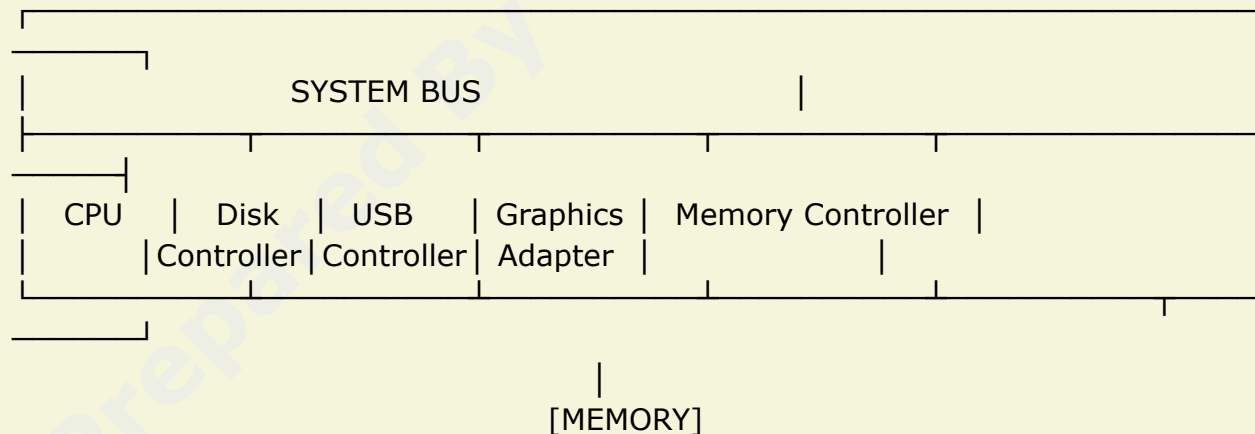
The cycle: OS relinquishes CPU control → user program executes → after its time quantum, CPU returns to OS → OS selects next process.

The kernel contains the most frequently used functions of the OS and is always resident in main memory. User programs and data occupy the remaining main memory.

6. Computer System Organization

6.1 Hardware Architecture

Visual: [Computer system bus architecture](#)



6.2 Device Controller

- Each I/O device has a corresponding device controller (hardware component).
- **A device controller:**
 - Maintains local buffer storage (small on-device cache).
 - Has special-purpose registers for control signals.
 - Moves data between the peripheral device and its local buffer.
 - One controller can manage multiple devices (e.g., USB hub).
- The OS device driver understands the device controller and provides the rest of the OS with a uniform interface.

6.3 CPU and Device Controller Parallelism

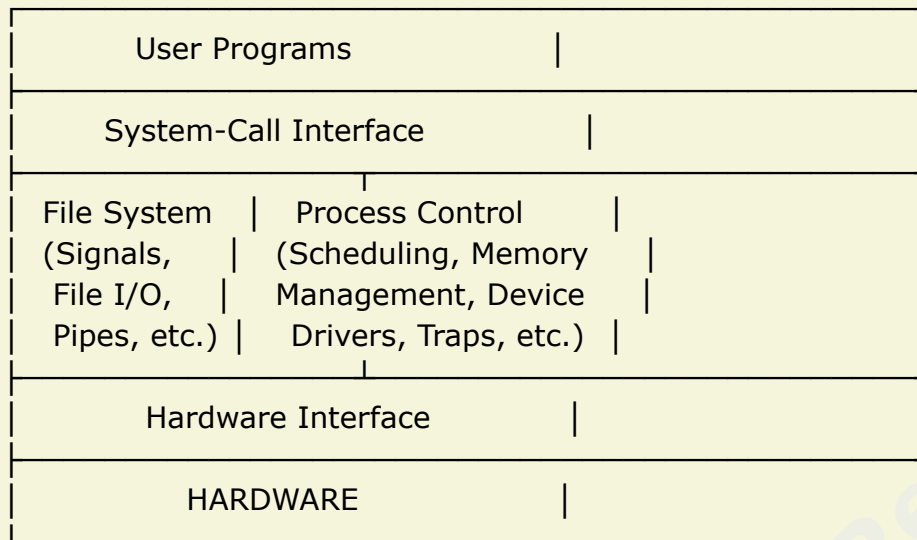
- CPU and device controllers can execute in parallel, competing for memory cycles.
 - A memory controller synchronizes access to shared memory to ensure orderly access.
-

7. OS Structure

7.1 Monolithic Structure

- **Definition:** The entire OS runs as a single program in kernel mode. All OS components (file system, device drivers, scheduler, memory manager) are combined into one large executable.
- **Examples:** Traditional UNIX, Linux, Windows.
- **Advantage:** Very high performance — no overhead from inter-module calls.
- **Disadvantage:** Difficult to maintain; a bug in any component can crash the entire system.

7.2 UNIX Traditional Structure

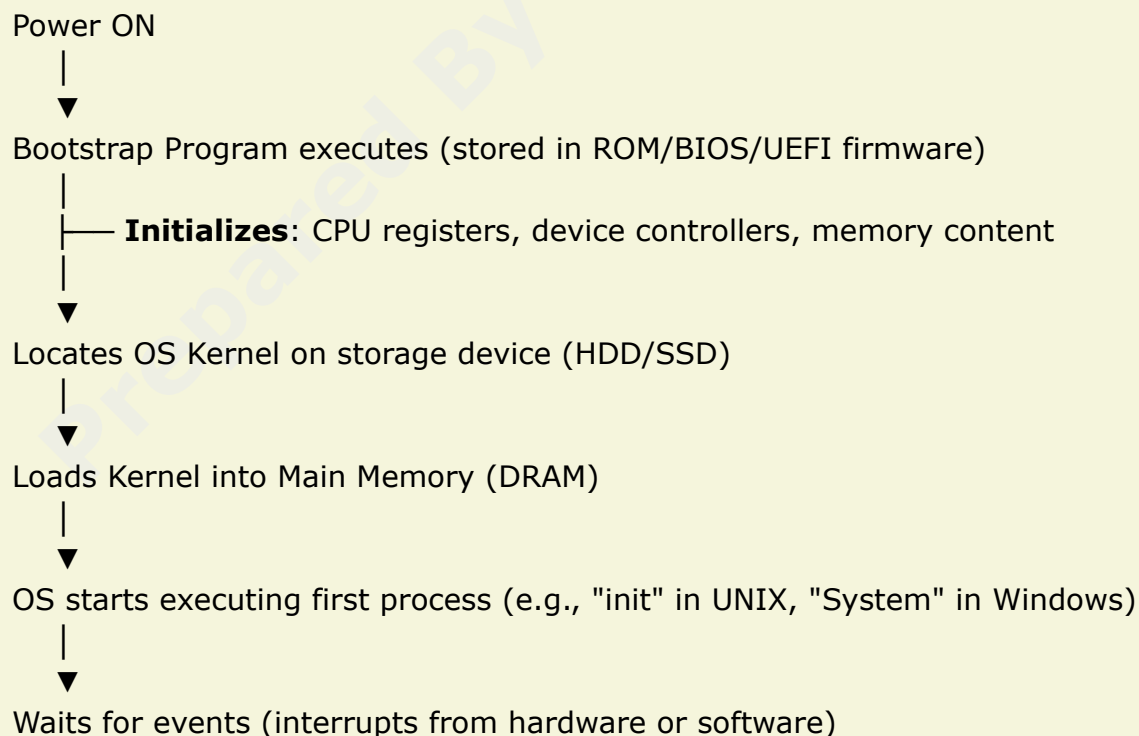


7.3 Android Architecture

Layers (bottom to top): Linux Kernel → Hardware Abstraction Layer → Android Runtime (ART) + Native C/C++ Libraries → Java API Framework → System Apps.

8. Booting a Computer

8.1 Boot Sequence (Step by Step)



8.2 Boot Types

Type	Definition
Cold Boot	Computer was completely switched off; fully powered on from scratch
Warm Boot	Using the OS to restart the computer (e.g., Start → Restart); hardware not fully powered off

8.3 Interrupts

- **Hardware interrupt:** Hardware device sends a signal to the CPU (usually via the system bus) at any time to request attention.
- **Software interrupt (Trap / System Call):** Software triggers an interrupt by executing a special operation called a system call (also called monitor call).
- On receiving an interrupt, the CPU suspends current execution and transfers control to the appropriate interrupt service routine (ISR).

Interrupt Handling Steps (Trap/Interrupt Sequence)

1. Hardware saves Program Counter (onto stack or special register)
2. Hardware loads new PC → identifies which interrupt occurred
3. Assembly language routine saves all registers
4. Assembly language routine sets up a new stack
5. Assembly language calls C code to run the service routine
6. Service routine calls the scheduler
7. Scheduler selects the next process to run (may be the interrupted one)
8. Assembly language routine loads PC & registers for the selected process
→ new process resumes execution

9. System Calls

9.1 Definition & Purpose

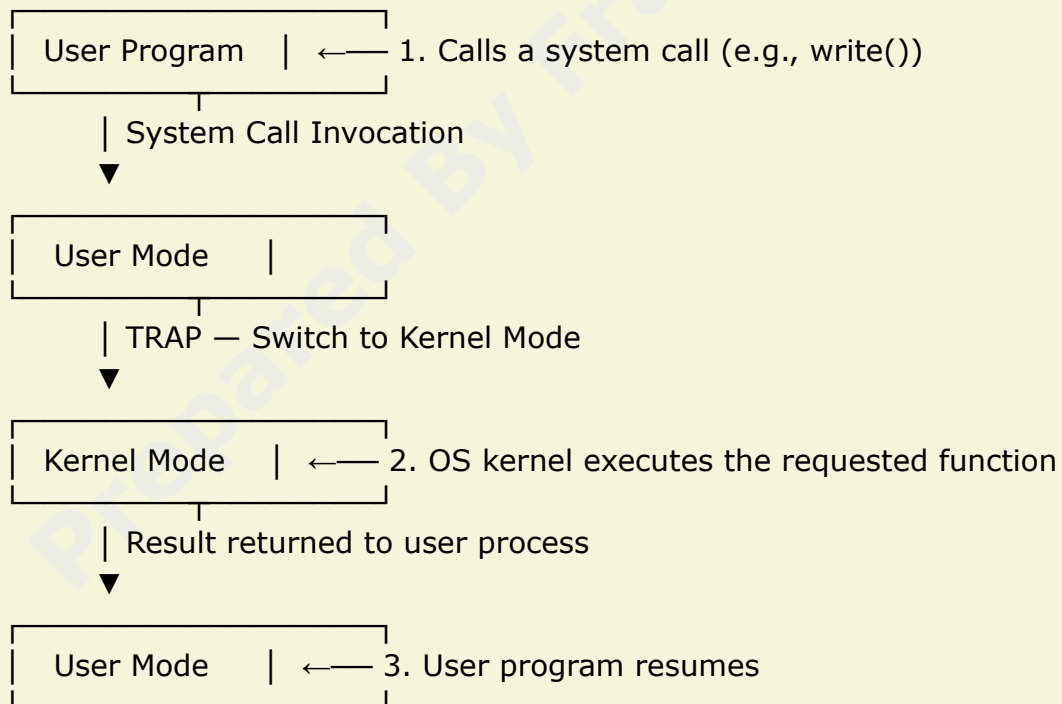
System Call: A programmatic interface (request) through which a user-level process asks the kernel to perform a privileged operation on its behalf.

- They enable interaction between user programs and hardware.
- They ensure security and controlled access to system resources.
- **They provide essential services:** file I/O, memory management, process control.

9.2 Key Concepts

Concept	Definition
Kernel Mode	Privileged CPU mode; kernel has full access to hardware and all system resources
User Mode	Restricted CPU mode; user processes cannot directly access hardware or kernel memory
Trap	Synchronous software interrupt that switches the CPU from User Mode → Kernel Mode to service a system call
File Descriptor	An integer that represents an open file or I/O device within a process (0=stdin, 1=stdout, 2=stderr)

9.3 System Call Workflow



9.4 Types of System Calls

Category	Purpose	UNIX Examples	Windows Examples
Process Control	Create, terminate, load, execute processes	fork(), exec(), exit(), wait()	CreateProcess(), ExitProcess()
File Management	Open, read, write, close files	open(), read(), write(), close()	CreateFile(), ReadFile()
Device Management	Request/release devices; read/write devices	ioctl(), read(), write()	SetConsoleMode(), ReadConsole()
Information Maintenance	Get/set system time, date, process attributes	getpid(), alarm(), sleep()	GetCurrentProcessID(), SetTimer()
Communication	Create/delete pipes, send/receive messages	pipe(), msgsnd(), msgrcv()	CreatePipe(), SendMessage()

9.5 Example: write() System Call in C

```
#include <stdio.h>
#include <unistd.h>

int main() {
    char message[] = "Hello, System Call!\n";
    write(1, message, sizeof(message));
    // Argument 1: file descriptor (1 = stdout)
    // Argument 2: pointer to data buffer
    // Argument 3: number of bytes to write
    return 0;
}
```

write() Call — Step-by-Step Kernel Execution

1. User Process → invokes write(fd, buffer, size)
2. CPU traps → switches to Kernel Mode
3. Kernel's write() handler invoked
4. Check Permissions & Validate Arguments (fd, buffer, size)
5. Find File Descriptor → look up in process's file descriptor table
6. Access File/Device associated with that fd (via file system or device driver)
7. Copy data from user buffer → kernel buffer (protects kernel from user data)
8. Write data from kernel buffer → file/device
9. Update File Metadata (modification time, file size)
10. Return status (bytes written, or error code if failed)
11. CPU returns to User Mode → user process resumes

File Descriptors:

FD Value	Default Binding
0	Standard Input (stdin) — e.g., keyboard
1	Standard Output (stdout) — e.g., screen
2	Standard Error (stderr) — e.g., screen

9.6 Tracing System Calls with strace (Linux)

Install strace (Ubuntu 24.04 comes with it pre-installed)
sudo apt install strace

Trace all system calls made by 'ls' command
strace ls

Count system calls and their timing
strace -c ls

Output columns: % time, seconds, usecs/call, calls, errors, syscall
usecs/call = microseconds per individual system call invocation

9.7 Why Applications Are OS-Specific

- Each OS provides a unique set of system calls — applications compiled on one OS cannot execute on another.
- **Solutions:**
 1. Interpreted language (Python, Ruby) — interpreter available for multiple OSes.
 2. Virtual machine language (Java) — bytecode runs on JVM regardless of host OS.
 3. Standard API (POSIX) — compiler generates OS-specific binaries from portable source code.

10. Virtualization & Virtual Machines

10.1 Definition

Virtualization: A technology that abstracts the hardware of a single physical computer (CPU, memory, disk, NIC) into multiple separate virtual execution environments, creating the illusion that each environment runs on its own private computer.

Each virtual environment can run a different OS (e.g., Windows and Linux simultaneously on one machine).

10.2 Key Terminology

Term	Definition
Virtual Machine (VM)	A software emulation of a complete computer system; runs its own guest OS
Host OS	The OS running on the physical (real) hardware
Guest OS	The OS running within a virtual machine
Hypervisor	Software that creates, manages, and isolates virtual machines; intermediary between host OS and VMs
Emulation	Simulating one CPU architecture on a different CPU architecture in software

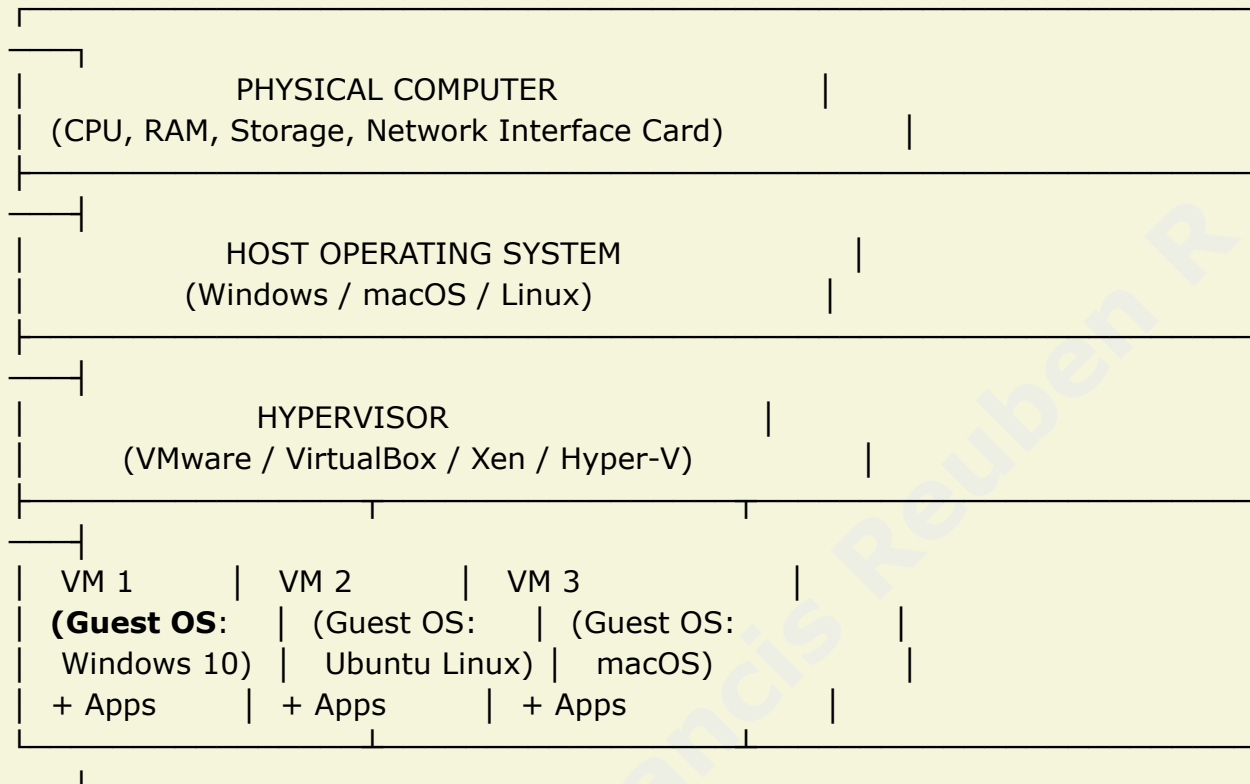
10.3 Virtualization vs Emulation

Aspect	Virtualization	Emulation
CPU architecture	Same on host and guest	Different on host and guest
Performance	Near-native	Significantly slower
Example	VMware running Windows on a Windows host	Apple Rosetta running IBM PowerPC apps on Intel x86

10.4 Benefits of Virtualization

Benefit	Detail
Resource Consolidation	Multiple OSes and applications on one physical machine
Isolation	VMs are isolated from each other → enhanced security
Flexibility	Easy to create, clone, snapshot, and migrate VMs
Cost-Effectiveness	Reduces hardware procurement costs
Testing & Development	Test software in different OS environments without separate hardware

10.5 Architecture: Computer with Single OS and 3 VMs



10.6 Historical Context

- IBM mainframes first used virtualization for multiple-user concurrent task execution.
 - VMware created virtualization as a *Windows application* — ran one or more guest copies of Windows (or other x86 OSES) within a host Windows system.
 - Apple's Rosetta emulated IBM PowerPC on Intel x86 when Apple transitioned CPUs (2005); later Apple transitioned again to Apple Silicon (ARM architecture, late 2020).
-

UNIT II — PROCESS MANAGEMENT

11. What is a Process?

11.1 Formal Definitions

Definition 1: A process is a program in execution. **Definition 2:** A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources. **Definition 3:** An instance of a program running on a computer; an entity that can be assigned to and executed on a processor.

Program vs Process:

Aspect	Program	Process
Nature	Passive — static set of instructions on disk	Active — dynamic execution in memory
Existence	One copy on disk	Multiple instances possible
Resources	None allocated	CPU, memory, files, I/O allocated
State	None	Has current execution state

Example: Double-clicking Chrome on your desktop loads the Chrome program (from disk) and creates one or more Chrome processes (in memory, executing).

11.2 Components of a Process

Every process consists of these five components:

Component	Description
Text Section (Code)	Compiled machine code — executable instructions
Data Section	Global and static variables
Heap	Dynamically allocated memory (<code>malloc/new</code>)
Stack	Function call frames, local variables, return addresses
Program Counter (PC)	Register indicating address of next instruction to execute

11.3 CPU Burst vs I/O Burst

Burst Type	Definition	CPU State
CPU Burst	Period where process actively executes instructions	CPU busy (productive)
I/O Burst	Period where process waits for I/O completion	CPU idle (wasteful if single process)

Key insight: The OS exploits I/O bursts of one process to execute the CPU burst of another.

Process A: [CPU]—[I/O wait]——[CPU]—[I/O wait]—[CPU]

Process B: [CPU]————[CPU]————[CPU]

Timeline: A, B, A, B, A, B...

Process Types Based on Burst Pattern:

Type	Pattern	Examples
I/O-bound	Many short CPU bursts, long I/O waits	Interactive text editor, database queries
CPU-bound	Few long CPU bursts, rare I/O requests	Scientific simulation, video encoder, compiler

11.4 The Multiprogramming Model

- **Conceptual model:** 4 independent processes appear to run simultaneously.
- **Reality:** Only ONE process is active on CPU at any instant — that instant is just very short.
- CPU switches between processes rapidly (context switching) giving the illusion of parallelism.

Time →

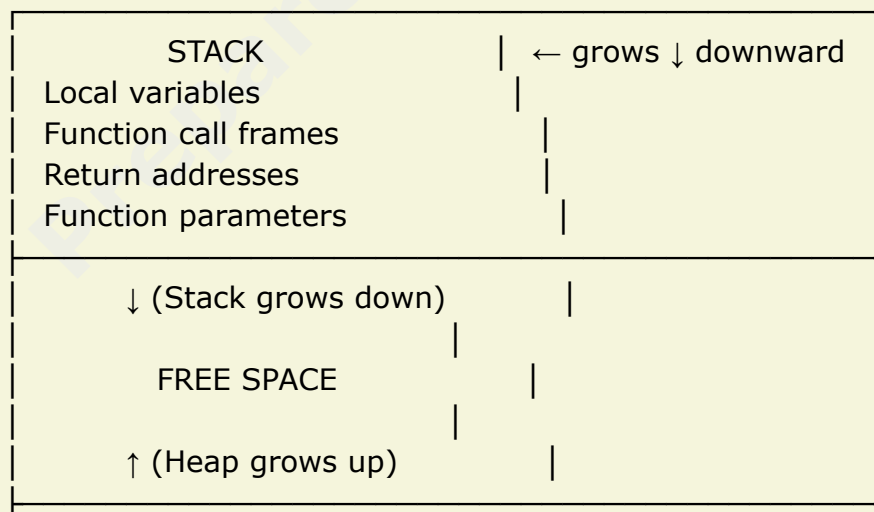
CPU actually: [A][A][B][B][C][C][A][A][D][D][B][B]...

Perceived: A, B, C, D all running "simultaneously"

12. Memory Layout of a Process

[Visual: Process memory layout](#)

High Memory Addresses



HEAP	← grows ↑ upward
Dynamically allocated memory (malloc, calloc, realloc, new)	
BSS SEGMENT	
Uninitialized global/static variables (automatically zeroed at program start)	
DATA SEGMENT	
Initialized global/static variables	
TEXT SEGMENT	← read-only
Compiled machine code instructions String literals and constants	

Low Memory Addresses

12.1 Segment Details

Segment	Contents	Grows	Permissions
Text	Machine code, string literals	Fixed	Read-only; shared across instances
Data	Initialized global/static variables	Fixed	Read-write
BSS	Uninitialized global/static variables	Fixed	Read-write; zero-initialized
Heap	malloc/new allocated memory	Upward (toward high addresses)	Read-write
Stack	Local vars, function frames, return addresses	Downward (toward low addresses)	Read-write

12.2 C Variable Placement Example

```
int init_global = 42;      // → DATA segment (initialized global)
int global_v;             // → BSS segment (uninitialized, zeroed at runtime)
const char *str = "hello"; // → TEXT segment (string literal)
```

```
int main() {
    int stack_v = 10;      // → STACK (local variable)
    int *heap_v = malloc(sizeof(int)); // → HEAP (dynamically allocated)
    static int static_local = 5;    // → DATA segment
}
```

12.3 Key Memory Concepts

Concept	Definition
Virtual Address Space	Each process has its own VA space — illusion of exclusive memory access. OS + MMU translate virtual → physical addresses.
Memory Management	OS responsible for allocating and deallocating memory to processes.
Dynamic Memory Allocation	Allocating memory at runtime using <code>malloc/free</code> (C) or <code>new/delete</code> (C++).
Stack Overflow	Stack grows beyond allocated limit — usually due to infinite/deep recursion or excessive local variable allocation.
Memory Leak	Dynamically allocated memory is never freed → gradual depletion of available heap memory.

12.4 Memory Layout of a C Program (Static vs Dynamic)

Static Layout (compile time): Dynamic Layout (runtime):

Text Segment	Heap (malloc/free)
Data Segment	Stack (function calls)
BSS Segment (zeroed at runtime)	

13. Process Control Block (PCB)

13.1 Definition

PCB (Process Control Block): Also called Process Table Entry or Task Control Block — a data structure created and maintained by the OS for every process. It contains sufficient information to interrupt a running process and later resume its execution as if no interruption had occurred.

The PCB is the key data structure enabling multiprocessing. The OS maintains one PCB per process.

Implementation (in C): PCB stored as a struct; **state** field implemented as an **enum**:

```
typedef enum { NEW, READY, RUNNING, BLOCKED, EXIT, SUSPENDED }  
ProcessState;  
struct PCB {  
    int  pid;           // Process Identifier  
    ProcessState state; // Current state  
    int  priority;      // Scheduling priority  
    void *program_counter; // Next instruction address  
    int  registers[];   // Saved CPU register values  
    void *memory_pointers; // Pointers to code/data/stack  
    // ... I/O info, accounting info, etc.  
};
```

13.2 Complete PCB Fields

Field	Description
Process Identifier (PID)	Unique integer; immutable throughout process lifetime; distinguishes from all other processes
State	Current execution state: New / Ready / Running / Blocked / Exit
Priority	Numeric priority level relative to other processes; used by scheduler
Program Counter (PC)	Address of the next instruction to execute when the process next gets the CPU
Context Data (CPU Registers)	Values of all CPU registers (general-purpose, index, status flags) at last preemption

Memory Pointers	Pointers to code, data, and stack memory areas; pointers to shared memory blocks; OR pointer to page table
I/O Status Information	Outstanding I/O requests, I/O devices assigned to this process, list of open files
Accounting Information	CPU time used, I/O time, wall-clock time, time limits, files opened
Parent PID (PPID)	PID of the process that created this process
Signals	Pending signals and registered signal handlers
CPU Scheduling Information	Scheduling queue pointers, scheduling parameters

13.3 PCB Layout Table

PROCESS CONTROL BLOCK	
PROCESS MANAGEMENT	FILE MANAGEMENT
• Registers	• Root directory
• Program Counter	• Working (current) directory
• CPU Status Word	• File descriptors
• Stack Pointer	• User ID (UID)
• Process State	• Group ID (GID)
• Priority	• Effective UID (EUID)
• Process ID (PID)	
• Parent PID (PPID)	
• Signals	
• Process Start Time	
• Total CPU Usage	
MEMORY MANAGEMENT	
	• Pointers to text, data, stack segments, OR
	• Pointer to page table

13.4 PCB Changes on State Transition

Example: Process 1 transitions from Running → Blocked due to an I/O request:

PCB Field	Changed?	Reason
State	✓ Yes	Running → Blocked
Program Counter	✓ Yes	Saved: address of next instruction to execute when resumed
Context Data (Registers)	✓ Yes	All CPU register values saved completely
I/O Status Information	✓ Yes	New I/O request recorded; device assigned
Accounting Information	✓ Yes	CPU time used updated; I/O device accessed logged
Process Identifier (PID)	✗ No	PID is immutable for the entire life of the process

13.5 Process Queues

- **Ready Queue:** Queue of PCBs of processes in the Ready state, waiting for CPU allocation.
- **I/O Device Queues:** Separate queue for each I/O device — processes blocked waiting for that device.

Process Queue Lifecycle:

Created → Ready Queue

↓ (dispatcher selects)

Running

- └─ I/O Request → I/O Queue (blocked) → (I/O complete) → Ready Queue
- └─ Creates Subprocess → waits for it → (child terminates) → Ready Queue
- └─ Preempted (interrupt) → Ready Queue
- └─ Completes → Terminated (removed from all queues)

13.6 Viewing the Process Table

Linux: view process table

ps aux # All processes

top # Dynamic real-time view

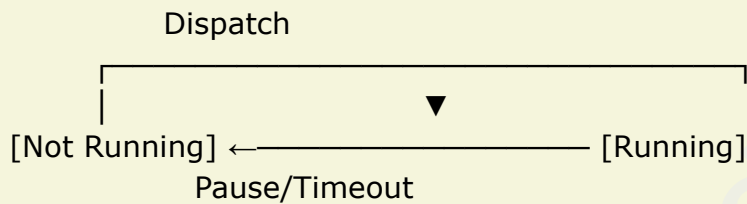
Linux kernel stores per-process info in: task_struct

Windows: managed by kernel object manager; view via Task Manager

14. Process State Models

14.1 Two-State Model

The simplest abstraction — a process is either using the CPU or it isn't.



State	Description
Running	Process is currently executing on the CPU
Not Running	Process is not executing (may be ready OR waiting for I/O)

Critical limitation: "Not Running" conflates two fundamentally different situations:

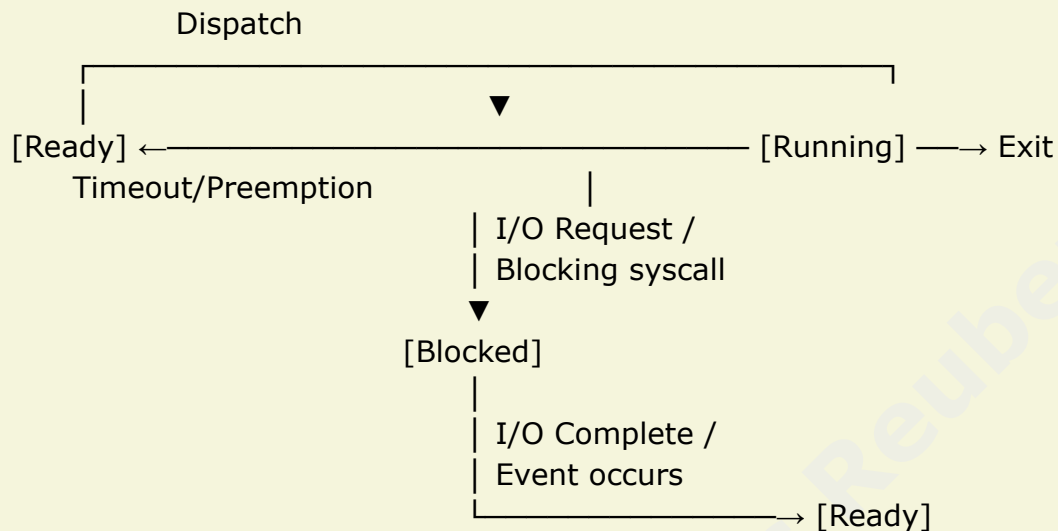
- Processes ready to run (just need CPU)
- Processes blocked on I/O (cannot run even if given CPU)

→ Dispatcher must scan the entire Not Running queue to find an unblocked process ($O(n)$ per scheduling decision) — inefficient.

Solution: Split "Not Running" into "Ready" and "Blocked" → Three-State Model.

14.2 Three-State Model

Splits "Not Running" into Ready and Blocked with separate queues.



State Definitions:

State	Description	Queue
Ready	Has all resources except CPU; waiting for dispatcher to assign CPU	Ready Queue
Running	Currently executing on the CPU; has all resources including CPU	On CPU
Blocked	Waiting for an event (I/O completion, signal, resource) — cannot run even if given CPU	Blocked Queue (per device)

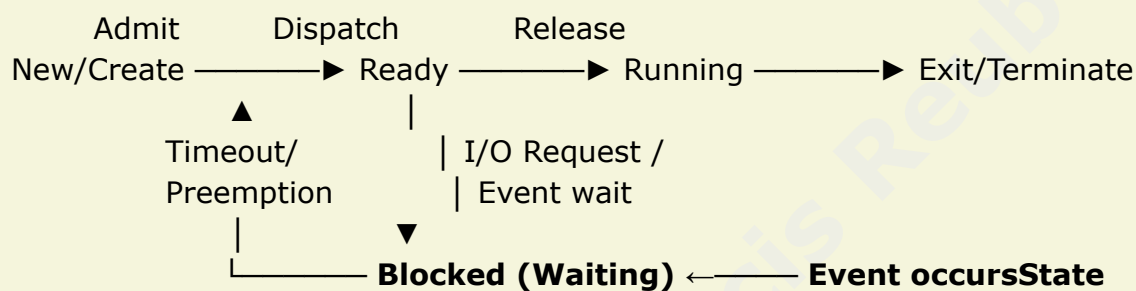
State Transitions:

Transition	Trigger
Ready → Running	CPU scheduler (dispatcher) selects this process
Running → Ready	Time slice expires (preemption); or higher priority process arrives

Running → Blocked	Process initiates I/O or makes a blocking system call
Blocked → Ready	I/O completes; event occurs; signal received
Running → Exit	Process terminates (calls <code>exit()</code>)

14.3 Five-State Model

Adds New and Exit states to better model the full process lifecycle.



Definitions (Five-State):

State	Definition
New / Create	Process has just been created; PCB allocated by OS, but resident set (code+data+stack) is NOT yet loaded into main memory. OS cannot run it yet.
Ready	Process is loaded into main memory; has all resources except CPU; waiting for dispatcher.
Running	Process is currently executing on the CPU. On a single CPU, at most one process can be in Running state at any time.
Blocked / Waiting	Process cannot execute until some event occurs (I/O completion, user input, signal). Can remain blocked for unlimited time — depends on external event.
Exit / Terminated	Process has finished execution. OS can extract audit info from PCB before releasing it. PCB may be kept briefly (zombie state) in case parent needs the exit status.

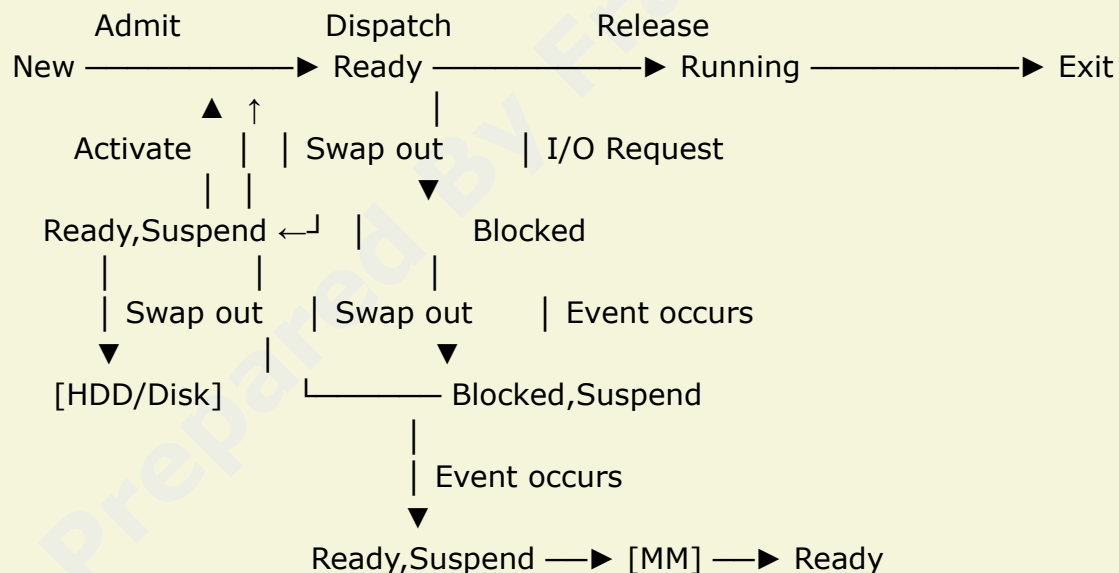
Exit State Utility:

- PCB retained temporarily after exit → allows same process to be re-created from already-loaded image (faster than reloading from HDD).
- Parent can call `wait()` to retrieve exit status; then OS fully removes the PCB.

Problem Being Solved

Also: Lack of free memory can prevent creation of new processes.

Suspend States



State	Definition
Ready, Suspend	Process is ready to run but has been swapped out of MM to disk. Waiting to be swapped back in when memory is available.
Blocked, Suspend	Process is waiting for an event AND is swapped out of MM. Must: (1) event occurs → move to Ready,Suspend, then (2) swapped into MM → Ready.

Suspend Reasoning:

- OS selects a lowest-priority blocked process, changes its state to Suspend, and moves its contents to disk.
- Space freed in MM is used to bring in other processes or create new ones.
- When the blocked event occurs for a suspended process, it is brought from disk directly into Ready state (bypassing Blocked state).
- Priority determines which process gets suspended — lowest priority processes are candidates.

Blocked vs Suspended:

- **Blocked:** Task is in MM, waiting for event.
- **Suspended:** Task has been swapped out to disk (task could have been in either Blocked or Ready state before suspension).

15. Process Creation & Termination

15.1 When is a Process Created?

Trigger	Description
System Initialization	When OS starts up, multiple processes are created (OS services, daemons)
System Call from Running Process	A currently running process calls <code>fork()</code> or <code>CreateProcess()</code> to create a child
User Request	User double-clicks an application or types a command in a shell
Batch Job Initiation	Long-term scheduler admits a batch job from the job pool

System Daemons: Background programs of the OS that run without direct user interaction.

Example (Linux): View daemons with `ps` command — shows PID, PPID, PGID, SID, UID, EUID, TTY.

15.2 Process Creation Details

- A parent process creates child processes which can in turn create their own children → forms a tree of processes.
- Each process is identified and managed via a Process Identifier (PID).

Resource Sharing Options:

Option	Description
Parent and children share all resources	Child inherits all parent's file descriptors, memory
Children share subset of parent's resources	Partial inheritance
Parent and child share no resources	Completely independent

Execution Options:

Option	Description
Concurrent	Parent and children execute simultaneously
Parent waits	Parent blocks until children terminate (using <code>wait()</code>)

UNIX Process Creation:

[illegible]

```

else {
    // Parent process
    wait(NULL); // Wait for child to complete
}

```

- **fork()**: Creates child as an exact duplicate of parent (same code, data, stack).
- **exec()**: Called after **fork()** in child to replace the process memory space with a new program — the child now runs a completely different program.

Address Space Options:

Option	Mechanism
Child is duplicate of parent	fork() — child gets copy of parent's address space
Child loads a different program	exec() called after fork()

15.3 Process Termination

When do Processes End?

Category	Type	Description
Voluntary	Normal exit	Process completes execution; calls exit()
Voluntary	Error exit	Process detects an error and calls exit() with error code
Involuntary	Fatal error	OS detects an unrecoverable error (e.g., illegal instruction, segfault, division by zero)
Involuntary	Killed by another process	Another process sends a kill signal (kill(pid, SIGKILL) in UNIX)

Process Termination System Calls

// Voluntary exit:

`exit(status_code);` // Returns status to parent

// Parent waits for child termination:

`pid_t pid = wait(&status);` // Returns child's PID and exit status

// Parent forcibly terminates child:

`abort();` // or `kill(child_pid, SIGKILL);`

Cascading Termination

- Some OSes do not allow a child to exist if its parent has terminated.
- When parent terminates → OS terminates all children, grandchildren, etc. (entire subtree).
- This termination cascade is initiated by the OS.

Zombie Process

- A process that has terminated but whose PCB has not yet been removed because the parent has not called `wait()`.
- The zombie occupies a PCB entry but no memory/CPU resources.

Orphan Process

- A process whose parent has terminated without calling `wait()`.
- In UNIX, orphans are re-parented to the `init` process (PID 1) which periodically calls `wait()`.

15.4 Process Hierarchy

OS	Hierarchy Model
UNIX/Linux	Tree of processes — process group. If a process exits, children are inherited by the exiting process's parent (or by <code>init</code>).
Windows	No concept of process hierarchy. All processes are created equal; parent-child relationship exists only at creation time.

16. Threads & Multithreading

16.1 What is a Thread?

Thread: The smallest unit of CPU execution within a process. Also called a Lightweight Process (LWP).

Process A

- |— Thread 1 [PC, Registers, Stack]
- |— Thread 2 [PC, Registers, Stack]
- |— Thread 3 [PC, Registers, Stack]

Shared: Address Space, Code, Data, Files, I/O resources

Example: A web browser has: one thread loading a webpage, another handling user input, another rendering the page.

16.2 What a Thread Has vs What it Shares

Owned per Thread (Private)	Shared among all Threads in a Process
Program Counter (PC)	Address space (entire virtual memory)
CPU Register set	Code (Text segment)
Stack & Stack Pointer	Data segment (global variables)
Thread ID	Open files and file descriptors
State	Child processes
Thread Control Block (TCB)	Signals and signal handlers
	Accounting information

16.3 Why Are Threads Needed?

Reason	Detail
Parallelism	Multiple threads can run on multiple CPU cores simultaneously
Single app doing many things	A word processor: one thread reads keyboard, one formats document, one writes to disk
Simpler programming model	Easier to structure complex applications
Less waiting	Threads overlap computation and I/O
Faster to create/destroy	No separate address space needed → cheaper than <code>fork()</code>
Improve responsiveness	Interactive app stays responsive while one thread does background work
Resource sharing	Threads share memory — no expensive IPC needed for data exchange
Economy	Less memory; less context switch overhead than full process switch
Scalability	Automatically benefits from multi-core CPUs

16.4 Each Thread Has Its Own Stack

Process Memory Layout (3 threads)

Thread 3's Stack	
Thread 2's Stack	
Thread 1's Stack	
HEAP	← Shared
DATA / BSS	← Shared
TEXT	← Shared

Key rule: Every thread has its own stack — this is what allows each thread to have independent function call chains, local variables, and recursion depth.

16.5 User-Level vs Kernel-Level Threads

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed By	User-space library (e.g., POSIX pthreads, Java threads)	OS kernel
OS Awareness	OS sees only ONE thread (the process)	OS aware of each thread individually
Context Switch	Faster — no kernel mode switch needed	Slower — requires kernel mode switch
Parallelism	✗ Not true parallelism on multi-core (OS schedules single process)	✓ True parallelism — OS can schedule threads on different cores
Blocking Call Effect	One thread's blocking I/O call blocks the entire process	Only the calling thread blocks; others continue
Portability	✓ More portable (no OS-specific kernel changes needed)	✗ Less portable
Overhead	Low	Higher

16.6 Thread Implementation Approaches

Model	Description
User-level only	All thread management (creation, scheduling) in user space; kernel unaware. Fast but no true parallelism.
Kernel-level only	Kernel manages all threads directly. True parallelism; slower context switch.
Hybrid (M:N)	M user threads mapped to N kernel threads. Balances performance and parallelism.

16.7 Three Server Models Compared

Model	Concurrency	I/O Type	Difficulty
Thread model	✓ Parallelism	Blocking syscalls	Medium
Single-threaded process	✗ No parallelism	Blocking syscalls	Easy
Finite-state machine	✓ Parallelism	Non-blocking syscalls + Interrupts	Hard

16.8 Process vs Thread — Complete Comparison

Aspect	Process	Thread
Use case	Essentially unrelated tasks	Parts of the same job
Address space	Own address space + PCB	Shares process's address space
Protection	Protected from other processes	No protection among threads of same process
Context switch	Done at kernel level only — expensive (saves/restores full process state)	Can be done at user or kernel level — cheaper
Ownership	One or more users	Usually a single user
Memory	Separate code, data, stack	Shared code+data; own registers+stack
Creation overhead	High (<code>fork()</code> copies entire address space)	Low (no address space copy)
Synchronization	Via IPC (pipes, sockets, shared memory)	Via synchronization primitives (mutex, semaphore) on shared memory

16.9 Multithreaded Web Server Example

// DISPATCHER THREAD:

```
while (TRUE) {  
    getNextRequest(&buf); // Wait for incoming HTTP request  
    handoffWork(&buf);    // Hand off to a worker thread from thread pool  
}
```

// WORKER THREAD:

```
while (TRUE) {  
    waitForWork(&buf);           // Wait for dispatcher to hand off work  
    lookForPageInCache(&buf, &page); // Check if page is in cache  
    if (pageNotInCache(&page)) {  
        readPageFromDisk(&buf, &page); // Cache miss → read from disk  
    }  
    returnPage(&page);           // Send response to client  
}
```

Threading perspective:

- **Single-threaded:** All requests handled sequentially → severe delay under load.
 - **Multi-threaded (dispatcher+workers):** Dispatcher thread feeds work to pool of worker threads → true concurrent request handling → much better throughput.
 - **Concurrency:** Multiple worker threads process different requests in parallel on multiple CPU cores.
-

17. Interprocess Communication (IPC)

17.1 Definition & Motivation

IPC: A mechanism that allows processes to communicate and synchronize their actions in an OS.

Cooperating vs Independent Processes:

Type	Definition
Independent Process	Cannot affect or be affected by other processes; no data sharing
Cooperating Process	Can affect or be affected by other processes; shares data

Reasons for Process Cooperation:

Reason	Detail
Information Sharing	Multiple processes access common data (e.g., shared file)
Computation Speedup	Parallel subtask execution
Modularity	System divided into well-defined processes (microservice model)
Convenience	Performing multiple tasks simultaneously

17.2 Two Models of IPC

MODEL 1: Shared Memory

Process A —————> [Shared]
Process B <———— [Memory]

Both read/write a shared memory region directly

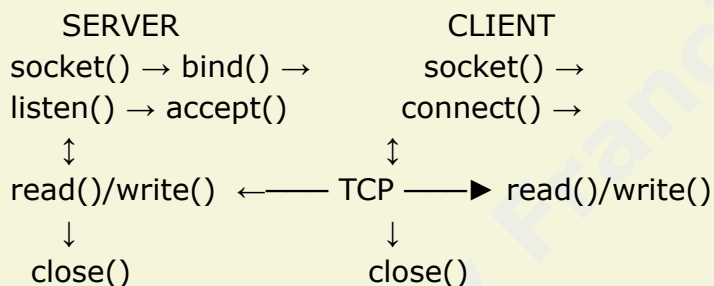
MODEL 2: Message Passing

Process A —> [Message] —> Process B

Processes exchange messages via OS-provided channels

Aspect	Shared Memory	Message Passing
Mechanism	Both processes map a common memory region	OS provides send/receive primitives
Speed	Fast — direct memory access after setup	Slower — each message involves OS system call
Synchronization	Programmer must synchronize explicitly	OS can handle synchronization
Use case	Large data exchange (high throughput)	Small messages; distributed systems
UNIX examples	<code>shmget()</code> , <code>shmat()</code>	<code>pipe()</code> , <code>msgsnd()</code> , <code>msgrcv()</code> , sockets

17.3 Socket Programming (TCP)



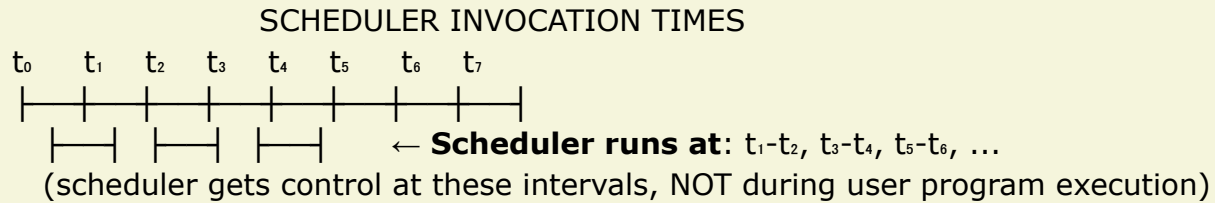
Example: A TCP server bound to port 8080 accepts connections; each client gets its own socket for bidirectional data exchange.

18. Types of Schedulers

18.1 Overview

The scheduler decides when to stop the currently executing process and selects the next process to run on the CPU. When switching processes, the OS:

1. Saves the current process's context into its PCB.
2. Loads the next process's context from its PCB.
3. Allows the new process to execute.



Scheduling Objectives:

- **Fairness:** Each process gets a fair share of CPU time.
- **No starvation:** No process waits indefinitely.
- **Efficient CPU use:** CPU kept productive as much as possible.
- **Low overhead:** Scheduler itself must execute quickly.

18.2 Three Types of Schedulers

Schedul er	Also Called	Operates On	Invocation Frequency	Key Function
Long-Term	Job Scheduler	Batch Queue (disk)	Infrequent (minutes)	Selects which jobs enter the Ready queue from the job pool
Medium-Term	Swapper	Suspended queues	Moderate	Swaps processes in/out of main memory
Short-Term	CPU Scheduler / Dispatcher	Ready Queue (MM)	Very frequent (milliseconds)	Selects which ready process runs on CPU next

18.3 Long-Term Scheduler

Detailed Function:

- Works with the batch queue (jobs waiting on disk).
- Selects next batch job to insert into the Ready queue.
- Admits new jobs into the system.
- Controls the degree of multiprogramming — how many processes are in the system simultaneously.
- **Primary objective:** Maintain a balanced mix of I/O-bound and CPU-bound processes (prevents CPU or I/O from being idle).

- Acts as the first level throttle on resource (CPU + MM) utilization.
- **Invoked:** When a completed job departs the system, making room for a new one.
- Frequency is low → can incorporate computationally intensive scheduling algorithms.
- **Note:** Not all systems have a long-term scheduler (many interactive systems do not).

18.4 Medium-Term Scheduler

Detailed Function:

- **Swaps out:** When a running process becomes blocked (I/O request / system call) and MM is full → removes process from MM to disk to free space.
- **Swaps in:** When ready queue is depleted, brings suspended-ready processes from disk back into MM.
- When a blocked event occurs for a suspended-blocked process → swaps it in from disk → directly into Ready state.
- **Invoked when:** Processes depart, or number of processes in Ready queue falls below a specified limit.
- Present only in OSes that support swapping.
- Balances the number of active processes in main memory.

Note: Medium-term scheduler also handles evicting lower-priority ready processes when many higher-priority tasks need to be accommodated in MM.

Memory: [P1 blocked] [P2 blocked] [P3 blocked] [P4 running]

↓ MM full, CPU starved

Medium-term scheduler: swap out P1 (lowest priority blocked) to disk

↓

Memory: [FREE SPACE] [P2 blocked] [P3 blocked] [P4 running]

New or ready-suspended process can now be loaded into freed space

18.5 Short-Term Scheduler

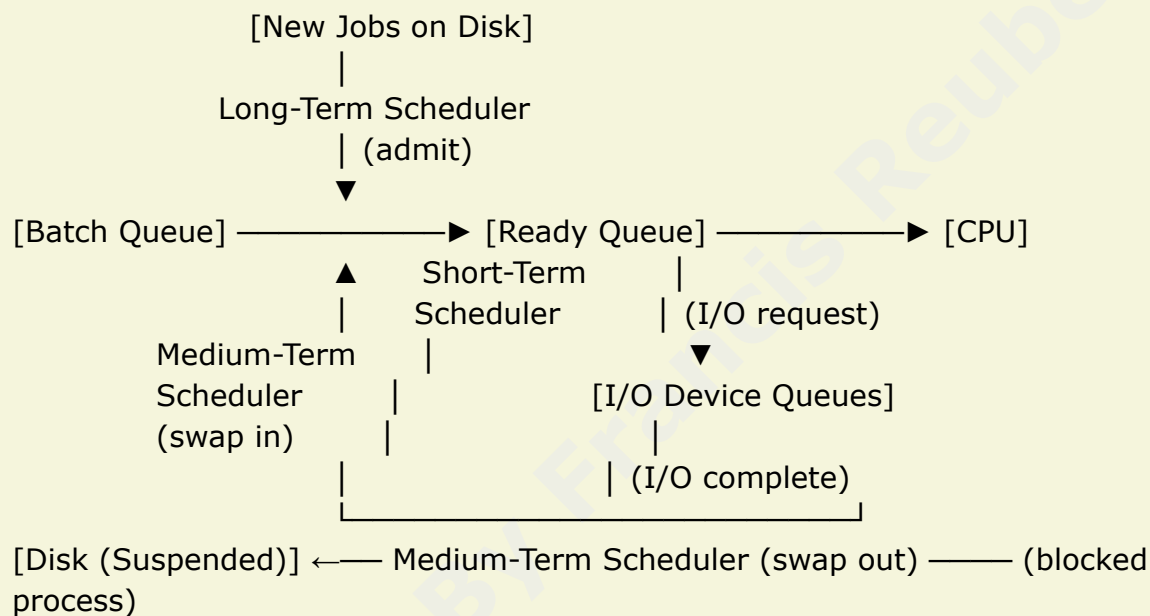
Detailed Function:

- Allocates CPU to one of the processes in the Ready queue that are resident in MM.
- Invoked for each process switch — must select the next process very quickly.
- **Invoked on events:**

- Time slice (quantum) completion — currently running process preempted.
- Interrupts and I/O completion — a blocked process becomes Ready.
- OS system calls invoked or completed.
- Sending or receiving signals.
- Clock ticks (timer-based interrupts).
- Activation of interactive programs.

Key Note: `task_struct` in Linux kernel holds per-process scheduling information.

18.6 Scheduler Interaction Diagram



Memory locations of queues:

- **Suspended queues:** maintained as OS data structures in main memory (only the resident-set of swapped-out processes is on HDD — not the queue structure itself).

19. Scheduling Criteria

19.1 The Five Performance Metrics

Criterion	Definition	Optimize
CPU Utilization	Percentage of time CPU is busy doing productive work	Maximize (target: 40–90%; 100% if context switch time negligible)
Throughput	Number of processes completing execution per unit time	Maximize
Turnaround Time	Total time from when a process first becomes ready to when it completes. = Exit time – Arrival time	Minimize
Waiting Time	Total time a process has spent waiting in the Ready queue (NOT blocked queue)	Minimize
Response Time	Time from when a job request is submitted (becomes ready) until the first response is produced (for interactive jobs)	Minimize

Why waiting time in the blocked queue is NOT counted:

- A process in the blocked queue is waiting for its own I/O request — the scheduler had no choice.
- Only the OS scheduler is responsible for time in the Ready queue.
- Time in Ready,Suspend queue IS counted (the scheduler decided to move the process there).

19.2 Key Formulae

Turnaround Time (TAT) = Exit Time – Arrival Time

Waiting Time (WT) = Turnaround Time – Burst Time
= TAT – Burst Time

Turnaround Time = Burst Time + Waiting Time

Response Time = Time of First Response – Arrival Time

CPU Utilization = (Total Busy Time / Total Time) × 100%

Throughput = Number of Processes Completed / Total Time
 Average TAT = Sum of all TATs / Number of processes
 Average WT = Sum of all WTs / Number of processes

19.3 Optimization Goals

Criterion	Direction	Notes
CPU Utilization	↑ Maximize	Range 0–100%; near 100% if context switch negligible
Throughput	↑ Maximize	Processes/time unit
Turnaround Time	↓ Minimize	Average value is usually optimized
Waiting Time	↓ Minimize	Average value is usually optimized
Response Time	↓ Minimize	Critical for interactive systems

Can all criteria be optimized simultaneously? Generally NO — trade-offs exist (e.g., reducing response time often increases average turnaround time). Scheduling algorithms typically optimize average values rather than individual process values.

20. Scheduling Algorithms

20.1 FCFS (First-Come, First-Served)

Definition

FCFS: Processes are assigned the CPU in the order they arrive in the Ready queue. Equivalent to a Round-Robin scheduler with Quantum = ∞ .

Key Properties:

- Processes run until they complete execution OR get blocked (I/O call / blocking syscall).
- Non-preemptive — OS cannot forcibly remove a running process.

- Favors processes that arrive early → unfair if early processes have long burst times.
- Starvation possible — later-arriving short processes must wait behind long processes.
- Not a fair scheduler.

FCFS — Worked Numerical

Given:

Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	3	6

Step 1 — Order of Execution (FCFS → by arrival time):

P1 → P2 → P3 → P4

Step 2 — Gantt Chart:

P1	P2	P3	P4	
0 5 8	16	22		

Step 3 — Completion Times:

Process	Burst	Arrival	Completion Time (CT)
P1	5	0	5
P2	3	1	8
P3	8	2	16
P4	6	3	22

Step 4 — TAT and WT:

Process	CT	AT	TAT = CT-AT	BT	WT = TAT-BT
P1	5	0	5	5	0
P2	8	1	7	3	4
P3	16	2	14	8	6
P4	22	3	19	6	13

Step 5 — Averages:

Average Turnaround Time = $(5 + 7 + 14 + 19) / 4 = 45 / 4 = 11.25$

Average Waiting Time = $(0 + 4 + 6 + 13) / 4 = 23 / 4 = 5.75$

Throughput = 4 processes / 22 time units = 0.182

CPU Utilization = $22/22 = 100\%$ (no idle gaps)

20.2 Round Robin (RR)

Definition

Round Robin: A preemptive scheduling algorithm where each process gets a fixed time quantum q on the CPU in circular order. After quantum expiry, the running process is moved to the end of the Ready queue and the next process is dispatched.

Key Properties:

- Preemptive — process is forcibly removed after time quantum.
- Designed for time-sharing environments.
- No starvation — every ready process eventually gets CPU time.
- Fair — all processes get equal CPU share.
- n processes running → each gets $1/n$ of CPU speed (illusion of exclusive but slower CPU).
- Context switching time must be $< 1\%$ of quantum time.

Effect of Quantum Size:

Quantum Size	Effect
Very large ($\rightarrow \infty$)	Degenerates to FCFS
Very small	High context-switch overhead; high responsiveness
Optimal	~80% of CPU bursts should be shorter than the quantum

Tie-breaking: When two processes arrive at the same time, the process with the lower ID enters the Ready queue first.

Process completion: When a process finishes early (before quantum expires), the scheduler immediately schedules the next process in the Ready queue.

RR Example 1 (Quantum = 4)

Given: P1: C1=8, P2: C2=12, both arrive at time 0.

Gantt Chart:

| P1 | P2 | P1 | P2 | P2 |
0 4 8 12 16 20

Process	Burst	Arrival	Completion	TAT	Wait
P1	8	0	12	12	4
P2	12	0	20	20	8

Avg TAT = $(12 + 20) / 2 = 16$

Avg WT = $(4 + 8) / 2 = 6$

Throughput = $2/20 = 0.1$

CPU Util = 100%

vs Non-preemptive (same processes):

Non-preemptive: Avg TAT = 14, Avg WT = 4

RR (q=4): Avg TAT = 16, Avg WT = 6

→ RR has higher TAT/WT but no starvation — both processes make progress.

RR Example 2 (Quantum = 3)

Given: P1: C1=8, P2: C2=12, both arrive at time 0.

Gantt Chart:

| P1 | P2 | P1 | P2 | P1 | P2 | P2 |
0 3 6 9 12 14 17 20

Proc ess	Bur st	Completi on	TA T	W ait
P1	8	14	14	6
P2	12	20	20	8

Avg TAT = $(14 + 20) / 2 = 17$

Avg WT = $(6 + 8) / 2 = 7$

Impact of reducing quantum from 4 → 3: Scheduler runs more often → TAT and WT increase → but tasks become more responsive.

RR Example 3 — Ready Queue Trace (Quantum = 2)

Rule: At each context switch, newly arrived processes enter the Ready queue before the preempted process (the arriving process gets into the queue first, then the preempted process is appended).

Given Process Table:

Process	Arrival	Burst
A	0	4
B	0	12
C	2	6
D	4	4
E	6	2

Timeline:

t=0: Ready=[A,B] → Run A(2)

t=2: C arrives; Ready=[B,C,A_preempted?] → A finishes 2 of 4 → Run B(2)

t=4: D arrives; A still has 2 left → Ready=[C,A,D] → Run C(2)

...

[**Full trace follows the rule:** newly arrived process enters before preempted process]

20.3 SJF (Shortest Job First) — Non-Preemptive

Definition

SJF: Out of all processes currently in the Ready queue, the CPU is assigned to the process with the smallest burst time. In case of tie, broken by FCFS (earlier arrival time wins). Non-preemptive — once a process starts running, it runs to completion.

Key Properties:

- Optimal average waiting time for a given set of processes (provably optimal among non-preemptive algorithms for minimizing average WT).
- Non-preemptive — currently running process cannot be interrupted.
- **Practical limitation:** Burst time of a process is generally not known in advance (must be estimated using techniques like exponential averaging).
- Ready queue is sorted by burst time (min-heap structure conceptually) — re-sorted whenever a new process arrives.
- In Ready queue, lowest burst time is at the head.

SJF Problem 1 — Full Worked Solution

Given:

Process	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Step 1 — Determine execution order:

At t=0: Only P4 has arrived → Run P4 (burst=6). No preemption → runs until t=6.

Ready Queue State:

t=0: [P4(6)] → Run P4

t=1: P2 arrives → queue (not yet running) = [P2(4)]

t=2: P5 arrives → queue = [P5(3), P2(4)]

t=3: P1 arrives → queue = [P1(1), P5(3), P2(4)]

t=4: P3 arrives → queue = [P1(1), P3(2), P5(3), P2(4)]

t=6: P4 finishes → queue = [P1(1), P3(2), P5(3), P2(4)]
→ Run P1 (burst=1, shortest)

t=7: P1 finishes → queue = [P3(2), P5(3), P2(4)]
→ Run P3 (burst=2)

t=9: P3 finishes → queue = [P5(3), P2(4)]
→ Run P5 (burst=3)

t=12: P5 finishes → queue = [P2(4)]
→ Run P2 (burst=4)

t=16: P2 finishes → all done

Gantt Chart:

| P4 | P1 | P3 | P5 | P2 |
0 6 7 9 12 16

Step 2 — Compute TAT and WT:

Proc ess	A T	B T	Ex it	TAT = Exit-AT	WT = TAT-BT
P4	0	6	6	6	0
P1	3	1	7	4	3
P3	4	2	9	5	3
P5	2	3	12	10	7
P2	1	4	16	15	11

Average TAT = $(6 + 4 + 5 + 10 + 15) / 5 = 40 / 5 = 8.0$

Average WT = $(0 + 3 + 3 + 7 + 11) / 5 = 24 / 5 = 4.8$

20.4 SRTF (Shortest Remaining Time First) — Preemptive SJF

Definition

SRTF (Shortest Remaining Time First): The preemptive version of SJF. At every moment (especially when a new process arrives or a time slice ends), the CPU is assigned to the process with the smallest remaining burst time. If the new arrival has a shorter remaining time than the currently running process, the current process is preempted.

Key Properties:

- Preemptive — running process can be interrupted when a shorter job arrives.
- Optimal minimum average waiting time among all scheduling algorithms for the same process set.
- **Tie-breaking:** FCFS (earlier arrival time wins).
- Waiting time is smaller for processes with short burst times (key advantage of SRTF).
- Practically difficult to implement (requires knowing/estimating remaining burst time).

SRTF Problem 2 — Full Worked Solution

Given:

Process	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Step-by-Step Execution (check for preemption on every arrival):

t=0: Arrived: P4(6). Queue: [P4(6)] → Run P4. Remaining: P4=6

t=1: P2 arrives(4). Running: P4(5 left). New: P2(4). $4 < 5 \rightarrow$ PREEMPT P4.

Queue: [P2(4), P4(5)] → Run P2

t=2: P5 arrives(3). Running: P2(3 left). New: P5(3). $3 = 3 \rightarrow$ tie $\rightarrow$ FCFS $\rightarrow$ P2 was running, keep P2.

Queue: [P2(3), P5(3), P4(5)]

Actually: $3 = 3$, both equal $\rightarrow$ keep current P2 running (FCFS: P2 arrived at $t=1$, P5 at $t=2$)

Run P2. Remaining: P2=3

t=3: P1 arrives(1). Running: P2(2 left). New: P1(1). $1 < 2 \rightarrow$ PREEMPT P2.

Queue: [P1(1), P2(2), P5(3), P4(5)] $\rightarrow$ Run P1

t=4: P3 arrives(2). Running: P1(0 left — P1 completes at $t=4!$).

P1 finishes at t=4. Queue: [P2(2), P3(2), P5(3), P4(5)]

P2 and P3 both have 2 remaining $\rightarrow$ tie $\rightarrow$ FCFS $\rightarrow$ P2 arrived earlier ($t=1$) $\rightarrow$ Run P2

t=6: P2 completes (ran from $t=4$ to $t=6$). Queue: [P3(2), P5(3), P4(5)] $\rightarrow$ Run P3

t=8: P3 completes. Queue: [P5(3), P4(5)] $\rightarrow$ Run P5

t=11: P5 completes. Queue: [P4(5)] $\rightarrow$ Run P4

t=16: P4 completes.

Gantt Chart:

|P4|P2|P2|P1|P2|P2|P3|P3|P5|P5|P5|P4|P4|P4|P4|P4|
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

Simplified:

|P4| P2 |P1| P2 | P3 | P5 | P4 |
0 1 3 4 6 8 11 16

Completion Times:

Proc ess	Exit Time
P4	16
P1	4
P3	8
P5	11
P2	6

Step 2 — Compute TAT and WT:

Process	AT	BT	Exit	TAT = Exit-AT	WT = TAT-BT
P4	0	6	16	16	10
P1	3	1	4	1	0
P3	4	2	8	4	2
P5	2	3	11	9	6
P2	1	4	6	5	1

Average TAT = $(16 + 1 + 4 + 9 + 5) / 5 = 35 / 5 = 7.0$

Average WT = $(10 + 0 + 2 + 6 + 1) / 5 = 19 / 5 = 3.8$

Key observation (SRTF property): P1 has the shortest burst time (1) → waiting time = 0. P2 has burst=4 → WT=1. Shorter burst → shorter wait.

20.5 Algorithm Comparison Table

Algorithm	Preemptive?	Starvation?	Overhead	Avg WT	Complexity	Best For
FCFS	✗ No	Possible (long jobs starve short ones)	Very Low	High	$O(1)$ scheduling	Batch systems
RR	✓ Yes	✗ No (by design)	Medium (context switches)	Higher than FCFS	$O(n)$ queue	Time-sharing, interactive
SJF	✗ No	Possible (long jobs may never run)	Low	Optimal non-preemptive	$O(n \log n)$ sort	Batch, known burst times

SRTF	✓ Yes	Possible (long jobs may starve)	High (preemption overhead)	Globally optimal	$O(n \log n)$ per arrival	Theoretical; optimal benchmark
------	-------	---------------------------------	----------------------------	------------------	---------------------------	--------------------------------

Additional Properties

Property	FCFS	RR	SJF	SRTF
Convoy effect	✓ Severe	✗ None	Mild	Mild
Response time	Poor	Good	Poor for long jobs	Best
Fairness	Based on arrival	Best	Unfair to long jobs	Unfair to long jobs
Implementation	Trivial	Simple	Requires known burst time	Requires known remaining time
Optimality	None	None	Optimal WT (non-preemptive)	Optimal WT (globally)

Convoy Effect (FCFS): One long CPU-bound process holds the CPU while many short I/O-bound processes wait → all I/O devices sit idle while short processes wait → then the short processes execute, finish quickly, and I/O devices are utilized again — but this repeats causing poor CPU and device utilization overall.

21. Key Formulae Quick Reference

21.1 Core Scheduling Formulae

Turnaround Time (TAT) = Completion Time (CT) – Arrival Time (AT)

Waiting Time (WT) = TAT – Burst Time (BT)
= CT – AT – BT

Response Time = First CPU Assignment Time – Arrival Time

Average TAT = $\Sigma(\text{TAT}_i) / n$

Average WT = $\Sigma(\text{WT}_i) / n$

Throughput = Number of Processes Completed / Total Elapsed Time

CPU Utilization (%) = (Total CPU Busy Time / Total Elapsed Time) × 100

21.2 RR-Specific

In RR with n processes and quantum q:

Each process gets $1/n \times \text{CPU speed}$ (perceived)

Context switch overhead must be < 1% of q

21.3 Quick Table: Solved Criteria Examples

From Lectures:

Non-preemptive, P1 arrives at 0 (burst=8), P2 arrives at 0 (burst=12):

Gantt: | P1(8) | P2(12) | Total time = 20
 0 8 20

Throughput = $2/20 = 0.1$ processes/time unit

CPU Util = 100%

Avg TAT = $(8 + 20) / 2 = 14$

Avg WT = $(0 + 8) / 2 = 4$

Non-preemptive, P1 arrives at 0 (burst=8), P2 arrives at 4 (burst=12):

Gantt: | P1(8) | P2(12) | Total time = 20
 0 8 20

Throughput = $2/20 = 0.1$

CPU Util = 100%

$$\begin{aligned}\text{Avg TAT} &= ((8-0) + (20-4)) / 2 = (8 + 16) / 2 = 12 \\ \text{Avg WT} &= ((0) + (8-4)) / 2 = (0 + 4) / 2 = 2\end{aligned}$$

Non-preemptive, P1 at 0(8), P2 at 4(12), P3 at 22(4):

Gantt: | P1(8) | P2(12) | idle(2) | P3(4) |
 0 8 20 22 26

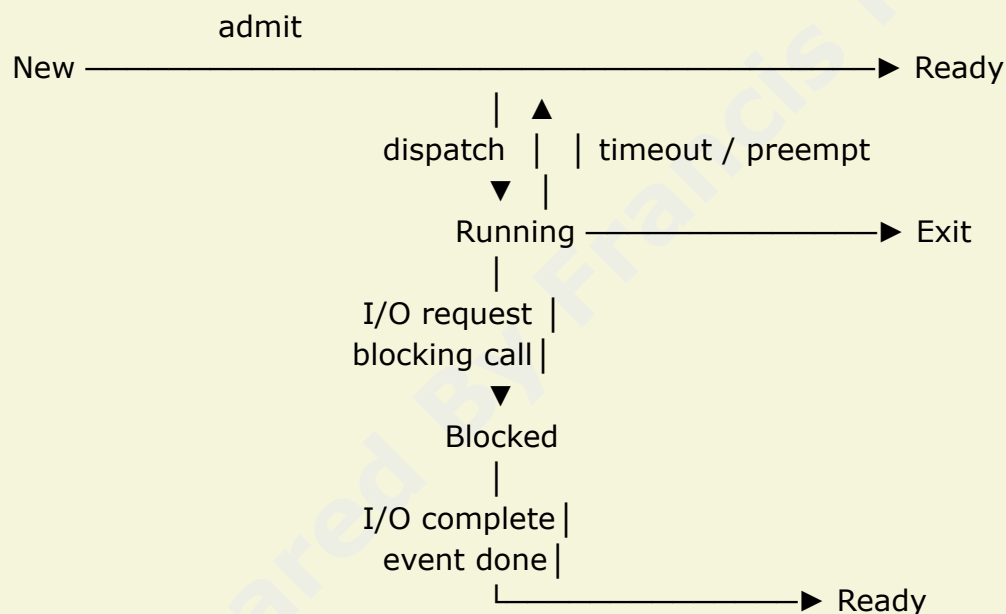
$$\text{CPU Util} = 24/26 = 92.3\%$$

$$\text{Throughput} = 3/26 = 0.115$$

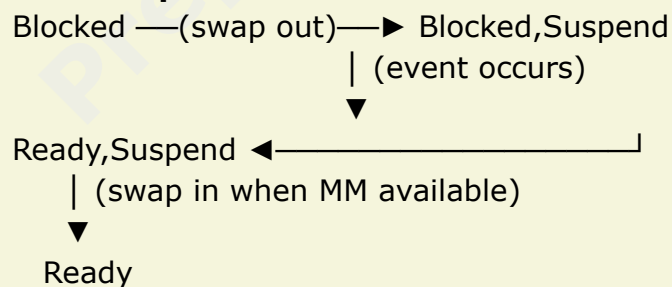
$$\text{Avg TAT} = ((8) + (16) + (4)) / 3 = 28/3 \approx 9.33$$

$$\text{Avg WT} = ((0) + (4) + (0)) / 3 = 4/3 \approx 1.33$$

21.4 Process State Transition Summary



With Suspend:



21.5 PCB Change Summary (State Transitions)

Transition	PCB Fields Changed
Running → Blocked	State, PC, Context Data, I/O Status Info, Accounting Info
Running → Ready	State, PC, Context Data
Ready → Running	State (only — other fields restored from PCB into CPU)
Any → Exit	State; resources released; PCB eventually removed
Ready → Ready,Suspend	State; resident set moved to disk
Blocked,Suspend → Ready,Suspend	State (event occurred)
Ready,Suspend → Ready	State; resident set reloaded into MM

End of OS Master Notes — Unit I & Unit II

Source: RV University CS1103 Lecture Materials, Dr. Sangeeta Bhanja Chaudhuri & Prof. Mouli Sankaran

Textbooks: Silberschatz et al. (OS Concepts, 10e) · Tanenbaum & Bos (Modern OS, 4e)

Operating Systems — Lab Master Notes (Labs 1–3)

RV University | CS1103 Lab | Semester II | CIE 1 Preparation

Covers: Lab 1 (Linux Commands + CLI Programs) · Lab 2 (Process System Calls) · Lab 3 (FCFS Scheduling Simulation)

Platform: Ubuntu Linux (GCC toolchain)

Table of Contents

LAB 1

1. Lab 1.1 — Basic Linux Commands Reference
 - 1.1.1 Navigation & File System Commands
 - 1.1.2 File Content & Search Commands
 - 1.1.3 System Information Commands
 - 1.1.4 Utility Commands
 - 1.1.5 Combined Shell Script Practice
2. Lab 1.2 — C Programs with Command Line Arguments
 - 1.2.1 argc and argv — Theory
 - 1.2.2 Program i — Sum of Numbers
 - 1.2.3 Program ii — Minimum of Numbers
 - 1.2.4 Program iii — Average of Numbers
3. Lab 1.3 — Printing Environment Variables

LAB 2 4. Lab 2 — Process Creation and Termination

- 2.1 Concepts: fork(), wait(), exit(), getpid()
- 2.2 Header Files Used
- 2.3 Complete Program: process_syscalls.c
- 2.4 Step-by-Step Code Walkthrough
- 2.5 Execution Flow Diagram
- 2.6 Output Analysis
- 2.7 Key Concepts Covered
- 2.8 Viva Questions & Answers

LAB 3 5. Lab 3 — FCFS CPU Scheduling Simulation

- 3.1 FCFS — Theory Recap
 - 3.2 Algorithm Steps
 - 3.3 Complete Program: fcfs.c
 - 3.4 Step-by-Step Code Walkthrough
 - 3.5 Worked Numerical Example
 - 3.6 Gantt Chart Construction
 - 3.7 Key Concepts & Viva Questions
6. Compilation & Execution Quick Reference
 7. Common Errors & Fixes
-

LAB 1.1 — BASIC LINUX COMMANDS REFERENCE

1.1.1 Navigation & File System Commands

pwd — Print Working Directory

Displays the absolute path of the current working directory.

Syntax: `pwd`

Example: `pwd`

Output: `/home/student/lab1`

ls — List Directory Contents

Displays information about files and directories in the current directory.

Syntax: `ls [options] [path]`

Example: `ls` # basic listing

`ls -l` # long format (permissions, size, date)

`ls -a` # show hidden files (starting with .)

`ls -lh` # long format with human-readable sizes

cd — Change Directory

Navigates between directories.

Syntax: `cd [path]`

Example: `cd /home/student` # absolute path

`cd Documents` # relative path (into Documents)

`cd ..` # move one level up

`cd ~` # go to home directory

`cd -` # go to previous directory

mkdir — Make Directory

Creates a new directory.

Syntax: `mkdir [options] dirname`

Example: `mkdir myproject`

`mkdir -p a/b/c` # create nested directories in one command

rmdir — Remove Directory

Removes an **empty** directory. Fails if directory has contents.

Syntax: `rmdir dirname`

Example: `rmdir myproject`

To remove non-empty directory: use `rm -r dirname`

cp — Copy Files / Directories

Copies files from one location to another.

Syntax: `cp [options] source destination`

Example: `cp file.txt /home/student/backup/`

`cp -r myfolder/ /tmp/myfolder_backup/` # -r for directories

mv — Move / Rename Files

Moves or renames files and directories.

Syntax: `mv source destination`

Example: `mv old_name.txt new_name.txt # rename`
`mv file.txt /tmp/ # move to /tmp`

rm — Remove Files

Permanently deletes files (no recycle bin).

Syntax: `rm [options] filename`

Example: `rm file.txt`

`rm -r mydir/ # recursively delete directory + contents`
`rm -f file.txt # force delete without confirmation`

touch — Create Empty File / Update Timestamp

Creates a new empty file, or updates the access/modification timestamp of an existing file.

Syntax: `touch filename`

Example: `touch notes.txt # creates empty notes.txt`
`touch file1.c file2.c # create multiple files`

ln — Create Links (Shortcuts)

Creates hard or symbolic links to files.

Syntax: `ln [options] target linkname`

Example: `ln file.txt hardlink.txt # hard link`
`ln -s file.txt symlink.txt # symbolic (soft) link`

Link Type

Property

Hard link	Points to same inode; file data survives original deletion
Symbolic link	Points to filename/path; breaks if original is deleted

1.1.2 File Content & Search Commands

cat — Display File Contents

Concatenates and displays file contents to standard output.

Syntax: `cat filename`

Example: `cat notes.txt`

`cat file1.txt file2.txt > combined.txt # merge files`

grep — Search for Pattern in File/Output

Searches for a specific string or pattern using regular expressions.

Syntax: `grep "pattern" filename`

`command | grep "pattern" # pipe usage`

Example: `grep "Banana" fruits.txt`

`ps aux | grep "python" # find python processes`

Practical session example:

`echo -e "Apple\nBanana\nMango" > fruits.txt`

`grep "Banana" fruits.txt`

Output: Banana

Options:

- i ignore case
 - n show line numbers
 - v invert match (show lines NOT matching)
 - r recursive search in directories
-

sort — Sort File Contents

Sorts lines in a text file alphabetically or numerically.

Syntax: `sort [options] filename`

Example: `echo -e "Zebra\nApple\nMango" > names.txt`
`sort names.txt`

Output:

Apple

Mango

Zebra

Options:

-r reverse order

-n numeric sort

-u remove duplicate lines

wc — Word Count

Counts lines, words, and characters in a file.

Syntax: `wc [options] filename`

Example: `echo -e "Hello\nUbuntu\nLinux" > sample.txt`
`wc sample.txt`

Output: 3 3 20 sample.txt

↑ ↑ ↑

lines words characters

Options:

-l count lines only

-w count words only

-c count bytes/characters only

Example: Output `3 3 20 sample.txt` means: 3 lines, 3 words, 20 characters.

echo — Print to Terminal

Prints a string or text to standard output (terminal).

Syntax: `echo "text"`

`echo -e "text"    # -e enables escape sequences`

Example: `echo "Hello Ubuntu"`

Output: Hello Ubuntu

`echo -e "Line1\nLine2"    # \n = newline`

Output:

Line1

Line2

`echo -e "Apple\nBanana\nMango" > fruits.txt    # redirect to file`

Escape sequences with **-e**:

Sequence	Meaning
<code>\n</code>	Newline
<code>\t</code>	Tab
<code>\\</code>	Literal backslash

1.1.3 System Information Commands

uname — OS Information

Prints basic information about the operating system.

Syntax: `uname [options]`

Example: `uname                    # prints OS name: Linux`

`uname -a                    # all info: kernel, hostname, version, arch`

`uname -r                    # kernel release version`

whoami — Current Username

Displays the username of the currently logged-in user.

Syntax: `whoami`

Example: `whoami`

Output: priyanka (or current username)

ps — Process Status

Displays currently running processes in the terminal.

Syntax: `ps [options]`

Example: `ps` # processes for current terminal session

`ps aux` # all processes for all users (detailed)

`ps -ef` # full format listing

df — Disk Filesystem Usage

Reports disk space usage of filesystems.

Syntax: `df [options]`

Example: `df`

`df -h` # human-readable sizes (GB, MB, KB)

Output columns: Filesystem, Size, Used, Avail, Use%, Mounted on

man — Manual Pages

Opens the manual (documentation) page for any Linux command.

Syntax: `man command_name`

Example: `man ls` # opens manual for `ls`

`man grep` # opens manual for `grep`

Press `q` to quit the manual

Press `/text` to search within manual

Press `n` for next match, `N` for previous

1.1.4 Utility Commands

locate — Find File

Searches for a file by name in a pre-built file database.

Syntax: `locate filename`

Example: `locate sample.txt`

Note: database must be updated with: `sudo updatedb`

wget — Download Files from Internet

Downloads files from a URL (HTTP, HTTPS, FTP).

Syntax: `wget URL`

Example: `wget https://example.com/sample.txt`

Downloads sample.txt to the current directory

Options:

`-O output_name` save with a different filename

`-q` quiet mode (no output)

`-c` resume interrupted download

cal — Calendar

Displays a calendar in the terminal.

Syntax: `cal`

`cal month year`

Example: `cal` # current month

`cal 2 2026` # February 2026

Output: formatted calendar grid

whereis — Locate Command Binary

Finds the exact binary, source, and manual locations of a command.

Syntax: `whereis command_name`

Example: `whereis python`

Output: `python: /usr/bin/python3`

`whereis grep`

Output: `grep: /usr/bin/grep /usr/share/man/man1/grep.1.gz`

whereis vs locate:

Command	What it finds
<code>whereis</code>	Binary, source code, and man page locations of a command
<code>locate</code>	Any file matching the name in the database

1.1.5 Combined Shell Script Practice

This script exercises all the key commands from Lab 1.1 in sequence.

```
#!/bin/bash
```

```
# Lab 1.1 Practice Script
```

```
# Create a file with content
```

```
echo -e "Ubuntu\nLinux\nOperating System" > os.txt
```

```
# View file contents
```

```
cat os.txt
```

```
# Count words, lines, characters
```

```
wc os.txt
```

```
# Sort file contents alphabetically
```

```
sort os.txt
```

Search for "Linux" in the file
grep "Linux" os.txt

Show current logged-in user
whoami

Show disk usage in human-readable format
df -h

Show current month calendar
cal

Locate where the grep binary is installed
whereis grep

Expected outputs:

cat os.txt:
Ubuntu
Linux
Operating System

wc os.txt:
3 4 34 os.txt

sort os.txt:
Linux
Operating System
Ubuntu

grep "Linux" os.txt:
Linux

whoami:
student

whereis grep:
grep: /usr/bin/grep /usr/share/man/man1/grep.1.gz

Complete Linux Commands Summary Table

#	Comm and	Syntax	Purpose
1	ls	ls [options]	List directory contents
2	pwd	pwd	Print working directory
3	mkdir	mkdir dirname	Create directory
4	cd	cd path	Change directory
5	rmdir	rmdir dirname	Remove empty directory
6	cp	cp src dst	Copy files
7	mv	mv src dst	Move / rename files
8	rm	rm filename	Delete files
9	uname	uname -a	OS information
10	locate	locate filename	Find file in database
11	touch	touch filename	Create empty file
12	ln	ln [-s] target link	Create hard/soft link
13	cat	cat filename	Display file contents
14	clear	clear	Clear terminal screen
15	ps	ps [options]	Display processes
16	man	man cmd	Access command manual
17	grep	grep "text" file	Search in file/output
18	echo	echo "text"	Print to terminal

19	<code>wget</code>	<code>wget URL</code>	Download from internet
20	<code>whoami</code>	<code>whoami</code>	Display current user
21	<code>sort</code>	<code>sort filename</code>	Sort file contents
22	<code>cal</code>	<code>cal [month year]</code>	Display calendar
23	<code>whereis</code>	<code>whereis cmd</code>	Find command location
24	<code>df</code>	<code>df -h</code>	Disk filesystem details
25	<code>wc</code>	<code>wc filename</code>	Count lines/words/chars

LAB 1.2 — C PROGRAMS WITH COMMAND LINE ARGUMENTS

1.2.1 `argc` and `argv` — Theory

When a C program is executed from the terminal with arguments, the OS passes those arguments to `main()` through two parameters:

Parameter	Type	Description
<code>argc</code>	<code>int</code>	Argument Count — total number of arguments including the program name itself
<code>argv</code>	<code>char *[]</code>	Argument Vector — array of strings; <code>argv[0]</code> = program name, <code>argv[1]</code> = first user argument, etc.

Command: `./Add_num 1 2 3 4`

↑ ↑ ↑ ↑

`argc` = 5 (program name + 4 numbers)

`argv[0]` = `"./Add_num"`

```
argv[1] = "1"
argv[2] = "2"
argv[3] = "3"
argv[4] = "4"
```

Key function: `atoi()`

Converts a string (character array) representation of an integer into an actual `int`.

```
#include <stdlib.h>
int atoi(const char *str);
```

Example:

```
atoi("42") → 42 (integer)
atoi("123") → 123
atoi("abc") → 0 (invalid input returns 0)
```

Why `atoi` is needed: All `argv[]` values are C strings (`char *`). Arithmetic cannot be performed on strings — they must be converted to `int` first.

1.2.2 Program i — Sum of Numbers

Objective: Accept integer numbers as command-line arguments and compute their sum.

Complete Program: `Add_num.c`

```
#include <stdio.h>
#include <stdlib.h>      // for atoi()

/* Function prototype */
int Add_num(int nParams, char *argv[]);

int main(int argc, char *argv[])
{
    printf("Calculate the sum of numbers entered by the user!!!\n");

    /* Validate: at least one argument required */
```

```

if (argc < 2)
{
    printf("Error: Insufficient parameters!!!\n");
    printf("Give at least one or more valid integer numbers as parameters.\n");
    return -1;
}

int nParams = argc - 1; /* number of user-provided arguments */
int sum = 0;

/* Print all argv values for verification */
for (int i = 0; i < argc; i++)
{
    printf(" argv[%d] = %s\n", i, argv[i]);
}

sum = Add_num(nParams, argv);
return 0;
}

int Add_num(int nParams, char *argv[])
{
    char *pParam;
    int sum = 0;
    int num;

    for (int i = 1; i <= nParams; i++)
    {
        pParam = argv[i]; /* pointer to i-th argument string */
        num = atoi(pParam); /* convert string → integer */
        sum += num;
        printf("pParam = %s, sum = %d\n", pParam, sum);
    }
    printf("The sum of numbers given is: %d\n", sum);
    return sum;
}

```

Compilation and Execution

Using cc compiler:

```
cc Add_num.c
```

```
./a.out 1 2 3 4
```

Using gcc with named output:

```
gcc -o Add_num Add_num.c
```

```
./Add_num 1 2 3 4
```

Sample Output

Calculate the sum of numbers entered by the user!!!

```
argv[0] = ./Add_num
```

```
argv[1] = 1
```

```
argv[2] = 2
```

```
argv[3] = 3
```

```
argv[4] = 4
```

```
pParam = 1, sum = 1
```

```
The sum of numbers given is: 1
```

```
pParam = 2, sum = 3
```

```
The sum of numbers given is: 3
```

```
pParam = 3, sum = 6
```

```
The sum of numbers given is: 6
```

```
pParam = 4, sum = 10
```

```
The sum of numbers given is: 10
```

Line-by-Line Explanation

Line	Explanation
<pre>int nParams = argc - 1;</pre>	Subtract 1 because <code>argv[0]</code> is the program name, not a number
<pre>for (int i = 0; i < argc; i++)</pre>	Print loop starts at 0 to show <code>argv[0]</code> (program name)
<pre>for (int i = 1; i <= nParams; i++)</pre>	Sum loop starts at 1 to skip <code>argv[0]</code> (program name)
<pre>pParam = argv[i]</pre>	Copy pointer to i-th argument string

```
num = atoi(pParam)
```

Convert string argument to integer for arithmetic

```
sum += num
```

Accumulate sum

1.2.3 Program ii — Minimum of Numbers

Objective: Accept integers as command-line arguments and find the minimum value.

Complete Program: **Minimum.c**

```
#include <stdio.h>
#include <stdlib.h>    // for atoi()
#include <limits.h>    // for INT_MAX (required on Linux)

/* Function prototype */
int f_Min(int nParams, char *argv[]);

int main(int argc, char *argv[])
{
    printf("Find the minimum of the numbers entered by the user!!!\n");

    if (argc < 2)
    {
        printf("Error: Insufficient parameters!!!\n");
        printf("Give at least one or more valid integer numbers as parameters.\n");
        return -1;
    }

    int nParams = argc - 1;
    int min = 0;

    for (int i = 0; i < argc; i++)
    {
        printf(" argv[%d] = %s\n", i, argv[i]);
    }

    min = f_Min(nParams, argv);
    printf("The minimum of the numbers given is: %d\n", min);
}
```

```

    return 0;
}

int f_Min(int nParams, char *argv[])
{
    char *pParam;
    int min = INT_MAX; /* Initialize to largest possible integer */
    int num;

    for (int i = 1; i <= nParams; i++)
    {
        pParam = argv[i];
        num = atoi(pParam);

        if (min > num)
            min = num; /* Update minimum if current num is smaller */

        printf("The min now is %d\n", min);
    }
    return min;
}

```

Compilation and Execution

```

gcc -o Minimum Minimum.c
./Minimum 1 2 3 2 1

```

Sample Output

Find the minimum of the numbers entered by the user!!!

```

argv[0] = ./Minimum
argv[1] = 1
argv[2] = 2
argv[3] = 3
argv[4] = 2
argv[5] = 1

```

The min now is 1

The min now is 1

The min now is 1

The min now is 1

The min now is 1

The minimum of the numbers given is: 1

Critical Concept: `INT_MAX` from `<limits.h>`

`INT_MAX` is the maximum value a 32-bit signed integer can hold:
2,147,483,647.

Why initialize `min = INT_MAX`?

- Ensures that the first real number compared is always less than the initial `min`.
- Any number from the input will be $\leq \text{INT_MAX}$, so the first comparison always updates `min` correctly.
- Without this, if `min = 0` and all inputs are positive, the minimum would incorrectly remain 0.

$\text{INT_MAX} = 2,147,483,647 = 2^{31} - 1$

$\text{INT_MIN} = -2,147,483,648 = -2^{31}$

Both defined in `<limits.h>`

1.2.4 Program iii — Average of Numbers

Objective: Accept integers as command-line arguments and compute their arithmetic mean.

Complete Program: **Avg.c** (Corrected)

Note: The original lab document has a bug: variable **average** is used but declared as **avg**. The corrected version is below.

```
#include <stdio.h>
#include <stdlib.h>    // for atoi()

/* Function prototype */
float f_Avg(int nParams, char *argv[]);

int main(int argc, char *argv[])
{
    printf("Find the average of the numbers entered by the user!!!\n");

    if (argc < 2)
    {
        printf("Error: Insufficient parameters!!!\n");
        printf("Give at least one or more valid integer numbers as parameters.\n");
        return -1;
    }

    int nParams = argc - 1;
    float avg = 0;    /* CORRECTED: was 'average' in original */

    for (int i = 0; i < argc; i++)
    {
        printf(" argv[%d] = %s\n", i, argv[i]);
    }

    avg = f_Avg(nParams, argv); /* CORRECTED: was 'average = f_Avg(...)' */
    printf("The average of the numbers given is: %f\n", avg);
    return 0;
}

float f_Avg(int nParams, char *argv[])
{
```



```

char *pParam;
int sum = 0;
int num;

for (int i = 1; i <= nParams; i++)
{
    pParam = argv[i];
    num = atoi(pParam);
    sum += num;
    printf("pParam = %s, sum = %d\n", pParam, sum);
}

return (float)sum / nParams;  /* Cast to float for decimal result */
}

```

Compilation and Execution

```

gcc -o Avg Avg.c
./Avg 1 2 3 2

```

Sample Output

Find the average of the numbers entered by the user!!!

```

argv[0] = ./Avg
argv[1] = 1
argv[2] = 2
argv[3] = 3
argv[4] = 2
pParam = 1, sum = 1
pParam = 2, sum = 3
pParam = 3, sum = 6
pParam = 2, sum = 8
The average of the numbers given is: 2.000000

```

Calculation: $(1 + 2 + 3 + 2) / 4 = 8 / 4 = 2.0$

Important: Integer Division Bug

```

/* WRONG — integer division truncates decimal */
return sum / nParams;      /* 7/3 = 2, not 2.33 */

```

```
/* CORRECT — cast to float before dividing */  
return (float)sum / nParams; /* 7/3 = 2.333... */
```

Comparison of All Three CLI Programs

Feature	Add_num	Minimum	Avg
Function name	Add_num()	f_Min()	f_Avg()
Return type	int	int	float
Initialization	sum = 0	min = INT_MAX	sum = 0
Core operation	sum += num	if (min > num) min = num	sum += num
Output	Running total + final sum	Running min + final min	Final average only
Header needed	stdlib.h	stdlib.h, limits.h	stdlib.h

LAB 1.3 — PRINTING ENVIRONMENT VARIABLES

Theory: What Are Environment Variables?

Environment Variables are dynamic named values maintained by the OS shell, available to all processes. They store system-wide configuration: paths, user info, locale settings, etc.

Examples: `PATH` (executable search directories), `HOME` (user's home directory), `USER` (current username), `SHELL` (active shell path).

The Third Parameter: `envp[ ]`

The `main()` function accepts a third optional parameter `envp[ ]`:

```
int main(int argc, char *argv[], char *envp[])
```

Parameter	Type	Contents
<code>argc</code>	<code>int</code>	Argument count
<code>argv[]</code>	<code>char * []</code>	Command-line argument strings
<code>envp[]</code>	<code>char * []</code>	Environment variable strings (format: "NAME=VALUE")

`envp[ ]` is a NULL-terminated array — the last element is `NULL`, which is used as the loop termination condition.

Complete Program: `env_var.c`

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[], char *envp[])
{
    printf("Lab2: Print the envp[] contents ...\n");

    /* Loop until NULL terminator is reached */
    for (int i = 0; envp[i] != NULL; i++)
    {
        printf("%s\n", envp[i]);
    }

    return 0;
}
```

Compilation and Execution

```
gcc -o env_var env_var.c
./env_var
```

Sample Output

```
Lab2: Print the envp[] contents ...
SHELL=/bin/bash
SESSION_MANAGER=local/student-VirtualBox:@/tmp/.ICE-unix/1234
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
GTK_MODULES=gail:atk-bridge
PWD=/home/student/lab1
LOGNAME=student
HOME=/home/student
USER=student
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
... (many more)
```

Line-by-Line Explanation

Line	Explanation
<code>char *envp[]</code>	Array of pointers to strings; each string is "VARNAME=VALUE"
<code>envp[i] != NULL</code>	Loop condition: stop when NULL sentinel is reached
<code>printf("%s\n", envp[i])</code>	Print each environment variable string

Alternative method using `extern char **environ`:

```
#include <stdio.h>
extern char **environ;

int main() {
    for (int i = 0; environ[i] != NULL; i++)
        printf("%s\n", environ[i]);
    return 0;
}
```

LAB 2 — PROCESS CREATION AND TERMINATION

2.1 Concepts: `fork()`, `wait()`, `exit()`, `getpid()`

`fork()` — Create Child Process

System call that creates a new child process as an **exact duplicate** of the calling (parent) process.

```
#include <unistd.h>
pid_t fork(void);
```

Return values:

Return Value	Who receives it	Meaning
0	Child process	"I am the child"
> 0 (child's PID)	Parent process	"My child has PID = this value"
-1	Parent process	Error — child creation failed

What gets duplicated in the child:

- Program counter (child continues from the instruction after `fork()`)
- All variables (with same values at the moment of fork)
- Open file descriptors
- Memory layout (stack, heap, data, text — copy-on-write)

What is DIFFERENT in child:

- PID (new unique PID assigned)
- Parent PID (PPID = parent's PID)
- Return value from `fork()` (0 in child, child's PID in parent)

getpid() — Get Process ID

```
#include <unistd.h>
pid_t getpid(void);
```

Returns the **PID** of the calling process.

```
printf("My PID is: %d\n", getpid());
```

wait() — Wait for Child to Terminate

```
#include <sys/wait.h>
pid_t wait(int *status);
```

Suspends the calling (parent) process until **any one of its child processes** terminates. Returns the PID of the terminated child.

Parameter	Description
<code>&status</code>	Pointer to int where exit status of child is stored
Return value	PID of terminated child, or -1 on error

Why `wait()` is essential:

- Prevents **zombie processes** — a terminated child whose PCB remains in the process table because the parent hasn't collected its exit status yet.
- Without `wait()`: child terminates → becomes zombie → occupies PCB entry indefinitely.
- With `wait()`: parent collects child's exit status → kernel removes the child's PCB completely.

exit() — Terminate Process

```
#include <stdlib.h>
void exit(int status);
```

Terminates the calling process and returns `status` to the parent.

Constant	Value	Meaning
<code>exit(0)</code>	0	Normal / successful termination
<code>EXIT_SUCCESS</code>	0	Macro for successful exit
<code>EXIT_FAILURE</code>	1	Macro for unsuccessful exit
<code>exit(-1)</code>	-1	Error condition (non-zero = error)

`exit()` vs `return` from `main()`:

Aspect	<code>exit(status)</code>	<code>return status</code> from <code>main()</code>
Scope	Works from any function in the program	Only works when called from <code>main()</code>
Effect	Immediately terminates process, calls <code>atexit()</code> handlers, flushes stdio buffers	Same — when in <code>main()</code> , internally calls <code>exit()</code>
Use case	Child processes, error conditions in nested functions	Normal program completion

2.2 Header Files Used

Header File	Functions Provided
<code><stdio.h></code>	<code>printf()</code> , <code>scanf()</code> , standard I/O
<code><stdlib.h></code>	<code>exit()</code> , <code>EXIT_FAILURE</code> , <code>EXIT_SUCCESS</code> , <code>atoi()</code>
<code><sys/types.h></code>	Defines <code>pid_t</code> type (process ID type, typically <code>int</code>)


```
<sys/wait.h>  wait(), waitpid()
>
```

```
<unistd.h>    fork(), getpid(), getppid(), POSIX API
```

2.3 Complete Program: `process_syscalls.c`

```
// process_syscalls.c
// Demonstrates: fork(), wait(), exit(), getpid()

#include <stdio.h>      // printf()
#include <stdlib.h>     // exit(), EXIT_FAILURE
#include <sys/types.h>  // pid_t
#include <sys/wait.h>   // wait()
#include <unistd.h>     // fork(), getpid()

int main(int argc, char **argv)
{
    pid_t pid;

    /* Create child process */
    pid = fork();

    if (pid == 0)
    {
        /* ===== CHILD PROCESS ===== */
        printf("This is the child process and pid is %d\n", getpid());

        int i = 0;
        for (i = 0; i < 8; i++)
        {
            printf("%d\n", i);
        }

        exit(0); /* Child terminates successfully */
    }
    else if (pid > 0)
    {
        /* ===== PARENT PROCESS ===== */
        printf("This is the parent process and pid is %d\n", getpid());
    }
}
```

```
    int status;
    wait(&status);          /* Parent waits for child to finish */
    printf("Child is reaped\n"); /* Printed after child terminates */
}
else
{
    /* ===== fork() FAILED ===== */
    printf("Error in forking..\n");
    exit(EXIT_FAILURE);
}

return 0;
}
```

2.4 Step-by-Step Code Walkthrough

Step 1: Include Headers

```
#include <stdio.h>    // printf
#include <stdlib.h>    // exit
#include <sys/types.h> // pid_t
#include <sys/wait.h>  // wait
#include <unistd.h>    // fork, getpid
```

Each header enables specific system-level functions. Omitting any causes compilation warnings or errors.

Step 2: Declare `pid_t pid`

```
pid_t pid;
```

- `pid_t` is a data type defined in `<sys/types.h>` — typically an alias for `int`.
 - Stores the **return value of `fork()`** which will determine which process is parent and which is child.
-

Step 3: Call `fork()`

```
pid = fork();
```

At this exact instruction, the OS:

1. Creates a new process (child) as a copy of the parent.
2. Returns `0` to the child's execution.
3. Returns the child's PID (a positive integer) to the parent's execution.
4. Both processes continue from the **next instruction** after `fork()`.

Before `fork()`: One process

After `fork()`: Two processes running concurrently

Parent: `pid = child's PID` (e.g., 4489)

Child: `pid = 0`

Step 4: Child Process Block (`pid == 0`)

```
if (pid == 0)
{
    printf("This is the child process and pid is %d\n", getpid());
    int i = 0;
    for (i = 0; i < 8; i++)
        printf("%d\n", i);
    exit(0);
}
```

- `pid == 0` → this block executes **only in the child process**.
 - `getpid()` returns the child's unique PID (e.g., 4489).
 - Loop prints integers 0 through 7.
 - `exit(0)` terminates the child process with status 0 (success).
-

Step 5: Parent Process Block (`pid > 0`)

```
else if (pid > 0)
{
    printf("This is the parent process and pid is %d\n", getpid());
    int status;
```

```
wait(&status);
printf("Child is reaped\n");
}
```

- `pid > 0` → this block executes **only in the parent process**.
 - `getpid()` returns the parent's PID (e.g., 4488).
 - `wait(&status)` **blocks the parent** until the child terminates.
 - After `wait()` returns (child has terminated), parent prints "Child is reaped".
 - "Reaped" = parent has collected child's exit status → child PCB is removed → no zombie.
-

Step 6: Error Condition (`pid < 0`)

```
else
{
    printf("Error in forking..\n");
    exit(EXIT_FAILURE);
}
```

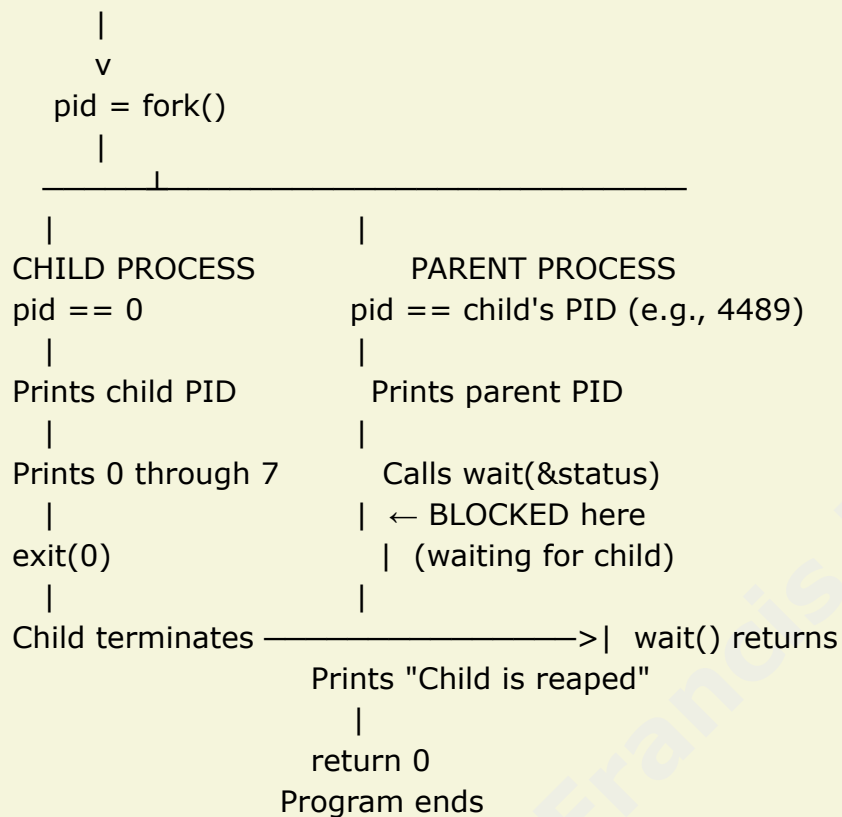
- If `fork()` returns `-1`, process creation failed (e.g., OS process limit reached, insufficient memory).
 - `EXIT_FAILURE` (= 1) signals abnormal termination.
-

Step 7: Compilation and Execution

```
gcc -o process_syscalls process_syscalls.c
./process_syscalls
```

2.5 Execution Flow Diagram

Program starts: main() called



2.6 Output Analysis

This is the parent process and pid is 4488

This is the child process and pid is 4489

0
1
2
3
4
5
6
7

Child is reaped

Why does parent print before child?

- Both parent and child are eligible to run after `fork()`.
- The OS scheduler decides which runs first.
- Output ordering is **non-deterministic** in general — parent may print before or after child.
- In this specific run: parent printed first (OS scheduled it first), then child ran, then parent resumed after `wait()` returned.

PIDs are sequential: 4488 (parent) and 4489 (child) because the OS assigns the next available PID to the new child.

2.7 Key Concepts Covered

Concept	Implementation	Notes
Process Creation	<code>fork()</code>	Creates exact duplicate; child has PID = 0 return value
Process Identification	<code>getpid()</code>	Returns calling process's unique PID
Process Termination	<code>exit(0)</code>	Child terminates successfully
Parent Waiting	<code>wait(&status)</code>	Parent blocks until any child terminates
Zombie Prevention	<code>wait()</code> in parent	Collects child exit status; kernel removes child PCB
Error Handling	<code>pid < 0</code> branch	Handles <code>fork()</code> failure gracefully

2.8 Viva Questions & Answers

Q1: What is `fork()`?

A: `fork()` is a UNIX system call that creates a new child process as an exact duplicate of the calling parent process. After `fork()`, both parent and child execute concurrently starting from the instruction after `fork()`.

Q2: What does `fork()` return?

A: Returns `0` in the child process; returns the child's PID (positive integer) in the parent process; returns `-1` if creation fails.

Q3: What is a zombie process?

A: A process that has completed execution (`exit()` called) but whose entry (PCB) still exists in the process table because the parent hasn't yet called `wait()` to collect its exit status. It consumes no CPU or memory, but occupies a process table slot.

Q4: Why do we use `wait()`?

A: (1) To prevent zombie processes — parent collects the child's exit status, allowing the OS to remove the child's PCB. (2) To synchronize parent and child — ensures parent doesn't proceed until the child finishes.

Q5: Difference between `exit()` and `return`?

A: `exit(status)` terminates the process from **any function**, flushes stdio buffers, and calls `atexit()` handlers. `return` from `main()` effectively calls `exit()` with the returned value — but `return` only works to exit the program when called from `main()`.

Q6: What is `pid_t`?

A: A data type (defined in `<sys/types.h>`) used to store process IDs. It is typically an alias for `int` on Linux systems.

Q7: What headers are required for `fork()` and `wait()`?

A: `fork()` requires `<unistd.h>`; `wait()` requires `<sys/wait.h>`; `pid_t` type requires `<sys/types.h>`.

Q8: Can both parent and child processes access the same variables?

A: After `fork()`, the child receives a **copy** of the parent's variables (copy-on-write). They do not share memory — modifications in one do not affect the other.

LAB 3 — FCFS CPU SCHEDULING SIMULATION

3.1 FCFS — Theory Recap

FCFS (First-Come, First-Served) is the simplest CPU scheduling algorithm. The process that **arrives first** in the Ready queue gets the CPU first. It follows the **FIFO (First-In, First-Out)** principle.

Key Properties:

- **Non-preemptive:** Once a process starts executing, it runs until it completes (or voluntarily blocks for I/O).
 - **No starvation:** Every process eventually gets the CPU.
 - **Convoy Effect:** A long process can cause many short processes to wait, reducing overall efficiency.
 - **Fair in arrival order:** Not fair in terms of burst time.
-

3.2 Algorithm Steps

FCFS Scheduling Algorithm:

INPUT: n processes with burst times $b[0..n-1]$, all arriving at $t=0$
(or sorted by arrival time if arrival times differ)

STEP 1: Set waiting time of first process: $w_time[0] = 0$

STEP 2: For each process i from 1 to $n-1$:
 $w_time[i] = \text{sum of burst times of all previous processes}$
 $= w_time[i-1] + b_time[i-1]$

STEP 3: For each process i from 0 to $n-1$:
 $tat[i] = b_time[i] + w_time[i]$
(Turnaround Time = Burst Time + Waiting Time)

STEP 4: Average Waiting Time = $(\text{sum of all } w_time[i]) / n$

Average Turnaround Time = (sum of all tat[i]) / n

STEP 5: Display results table and averages.

Formulae:

Waiting Time[0] = 0

Waiting Time[i] = $\Sigma(b_time[j])$ for j = 0 to i-1
= Waiting Time[i-1] + Burst Time[i-1]

Turnaround Time[i] = Burst Time[i] + Waiting Time[i]
= Completion Time[i] – Arrival Time[i]

Average WT = (WT[0] + WT[1] + ... + WT[n-1]) / n

Average TAT = (TAT[0] + TAT[1] + ... + TAT[n-1]) / n

3.3 Complete Program: fcfs.c

The lab document reuses the `process_syscalls.c` code header but gives the correct FCFS output. The actual FCFS C program implementing the algorithm is given below — this is the intended complete implementation.

```
// fcfs.c — FCFS CPU Scheduling Simulation
// Simulates First Come First Served scheduling
// Computes Waiting Time and Turnaround Time for each process
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
```

```
    /* ——— Step 1: Variable Declaration —————
    */
    int n;                // number of processes
    int b_time[20];        // burst time array
    int w_time[20];        // waiting time array
    int tat[20];          // turnaround time array
    int avg_wtime = 0;     // sum for average waiting time
```

```

int avg_tat  = 0;          // sum for average turnaround time
int i, j;                // loop variables

/* —— Step 2: Input Number of Processes ——
*/
printf("Enter total number of processes (maximum 20): ");
scanf("%d", &n);

/* —— Step 3: Input Burst Times
*/
printf("\nEnter Process Burst Time:\n");
for (i = 0; i < n; i++)
{
    printf("P[%d]: ", i + 1);
    scanf("%d", &b_time[i]);
}

/* —— Step 4: Calculate Waiting Time ——
*/
w_time[0] = 0;           // first process has no waiting time
for (i = 1; i < n; i++)
{
    w_time[i] = 0;
    for (j = 0; j < i; j++)
        w_time[i] += b_time[j]; // sum all previous burst times
}

/* —— Step 5: Calculate Turnaround Time ——
*/
for (i = 0; i < n; i++)
    tat[i] = b_time[i] + w_time[i];

/* —— Step 6: Accumulate Averages
*/
for (i = 0; i < n; i++)
{
    avg_wtime += w_time[i];
    avg_tat  += tat[i];
}

/* —— Step 7: Display Output Table ——
*/

```

```

printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (i = 0; i < n; i++)
{
    printf("P[%d]\t\t%d\t\t%d\t\t%d\n",
        i + 1, b_time[i], w_time[i], tat[i]);
}

/* —— Step 8: Print Averages —— */
printf("\nAverage Waiting Time:   %d\n", avg_wtime / n);
printf("Average Turnaround Time: %d\n", avg_tat / n);

return 0;
}

```

Compilation and Execution

```

gcc -o fcfs fcfs.c
./fcfs

```

3.4 Step-by-Step Code Walkthrough

Variable Declarations Explained

```

int n;           // total number of processes (max 20)
int b_time[20];  // array storing burst time of each process
int w_time[20];  // array storing computed waiting time per process
int tat[20];     // array storing computed turnaround time per process
int avg_wtime;   // accumulates sum of waiting times (for computing average)
int avg_tat;     // accumulates sum of turnaround times (for computing average)

```

Variable	Type	Purpose
n	int	Number of processes entered by user
b_time[20]	int[20]	Burst time for each process P[1]...P[n]
w_time[20]	int[20]	Waiting time for each process P[1]...P[n]
tat[20]	int[20]	Turnaround time for each process P[1]...P[n]
avg_wtime	int	Average waiting time
avg_tat	int	Average turnaround time

<code>w_time[20]</code>	<code>int[]</code>	Waiting time (computed) for each process
<code>tat[20]</code>	<code>int[]</code>	Turnaround time (computed) for each process
<code>avg_wtime</code>	<code>int</code>	Sum → divide by n → average waiting time
<code>avg_tat</code>	<code>int</code>	Sum → divide by n → average turnaround time

Waiting Time Calculation — Inner Loop Logic

```

w_time[0] = 0;           // P[1] waits 0 time (runs first)
for (i = 1; i < n; i++) // for P[2] through P[n]
{
    w_time[i] = 0;
    for (j = 0; j < i; j++)
        w_time[i] += b_time[j]; // sum burst times of all processes before i
}

```

Why this works: In FCFS (all arrive at $t=0$), process `i` waits for ALL processes before it to finish. So:

```

w_time[0] = 0           (no one before P[1])
w_time[1] = b_time[0]   (waits for P[1] to finish)
w_time[2] = b_time[0] + b_time[1] (waits for P[1] and P[2])
w_time[3] = b_time[0] + b_time[1] + b_time[2]

```

Optimization — $O(n)$ version:

// The nested loop is $O(n^2)$. This equivalent $O(n)$ version is more efficient:

```

w_time[0] = 0;
for (i = 1; i < n; i++)
    w_time[i] = w_time[i-1] + b_time[i-1];

```

Both versions produce identical results; the $O(n)$ version is preferred.

Turnaround Time Calculation

```
for (i = 0; i < n; i++)  
    tat[i] = b_time[i] + w_time[i];
```

Formula: $TAT = \text{Burst Time} + \text{Waiting Time}$

This is equivalent to $\text{Completion Time} - \text{Arrival Time}$ (when arrival time = 0 for all).

Average Calculations

```
for (i = 0; i < n; i++)  
{  
    avg_wtime += w_time[i];    // sum all waiting times  
    avg_tat   += tat[i];       // sum all turnaround times  
}  
// Divide sums by n to get averages:  
printf("Average Waiting Time:   %d\n", avg_wtime / n);  
printf("Average Turnaround Time: %d\n", avg_tat / n);
```

3.5 Worked Numerical Example

Input:

Number of processes: 4
P[1]: Burst = 5
P[2]: Burst = 8
P[3]: Burst = 2
P[4]: Burst = 7

Step-by-Step Calculation Table

Waiting Time:

Process	Calculation	Waiting Time
P[1]	0	0
P[2]	b_time[0] = 5	5
P[3]	b_time[0] + b_time[1] = 5 + 8	13
P[4]	b_time[0] + b_time[1] + b_time[2] = 5 + 8 + 2	15

Turnaround Time = Burst + Waiting:

Process	Burst	Waiting	Turnaround
P[1]	5	0	5
P[2]	8	5	13
P[3]	2	13	15
P[4]	7	15	22

Averages:

Average Waiting Time = $(0 + 5 + 13 + 15) / 4 = 33 / 4 = 8$

Average Turnaround Time = $(5 + 13 + 15 + 22) / 4 = 55 / 4 = 13$

Expected Program Output

Enter total number of processes (maximum 20): 4

Enter Process Burst Time:

P[1]: 5

P[2]: 8

P[3]: 2

P[4]: 7

Process	Burst Time	Waiting Time	Turnaround Time
---------	------------	--------------	-----------------

P[1]	5	0	5
P[2]	8	5	13
P[3]	2	13	15
P[4]	7	15	22

Average Waiting Time: 8

Average Turnaround Time: 13

3.6 Gantt Chart Construction

A **Gantt chart** is a horizontal bar chart representing the time-line of CPU allocation to processes.

FCFS Gantt Chart — Example (Bursts: P1=5, P2=8, P3=2, P4=7):

P[1]	P[2]	P[3]	P[4]	
0	5	13	15	22

Timeline:

- P[1]: runs from $t=0$ to $t=5$ (5 units)
- P[2]: runs from $t=5$ to $t=13$ (8 units)
- P[3]: runs from $t=13$ to $t=15$ (2 units)
- P[4]: runs from $t=15$ to $t=22$ (7 units)

How to draw manually:

1. Start at time = 0.
2. Place P[1] from 0 to $b_time[0] = 5$. Mark boundary at 5.
3. Place P[2] from 5 to $5+8 = 13$. Mark boundary at 13.
4. Place P[3] from 13 to $13+2 = 15$. Mark boundary at 15.
5. Place P[4] from 15 to $15+7 = 22$. Mark boundary at 22.

Reading the Gantt chart:

- **Completion time** of $P[i]$ = right boundary of its block.
- **Waiting time** of $P[i]$ = left boundary of its block (when all arrive at $t=0$).
- **Turnaround time** = right boundary – arrival time.

Second Example: Manual Calculation Practice

Input: P1(burst=4), P2(burst=3), P3(burst=5) — all arrive at $t=0$.

Gantt Chart:

```
| P1 | P2 | P3 |  
0   4   7   12
```

Table:

Process	BT	WT	TAT
P1	4	0	4
P2	3	4	7
P3	5	7	12

Avg WT = $(0 + 4 + 7) / 3 = 11 / 3 = 3.67$

Avg TAT = $(4 + 7 + 12) / 3 = 23 / 3 = 7.67$

3.7 Key Concepts & Viva Questions

Key Concepts

Concept	Detail
Non-preemptive	Running process cannot be interrupted; runs to completion
FIFO principle	Processes served in the order they arrive
Convoy effect	One long process delays all short processes behind it → poor CPU + I/O utilization
No starvation	Every process eventually gets the CPU (guaranteed progress)
Simple implementation	Requires only a FIFO queue; $O(n)$ computation
Not optimal	Does not minimize average waiting time (SJF is optimal)

FCFS Limitations vs Advantages

Aspect	FCFS
Average waiting time	Can be high (especially with convoy effect)
Fairness	Fair in arrival order; unfair based on burst time
Overhead	Minimal — no sorting or priority computation needed
Responsiveness	Poor for interactive systems
Implementation	Trivially simple

Viva Questions & Answers

Q1: What is FCFS scheduling?

A: FCFS (First-Come, First-Served) is a non-preemptive CPU scheduling algorithm that assigns the CPU to processes in the order they arrive in the Ready queue, following the FIFO principle.

Q2: Is FCFS preemptive or non-preemptive?

A: Non-preemptive. Once a process begins executing on the CPU, it runs until it voluntarily releases the CPU (completes execution or performs a blocking I/O call).

Q3: What is waiting time?

A: The total amount of time a process spends waiting in the Ready queue before it gets the CPU. In FCFS: $WT[i] = \text{sum of burst times of all processes that run before process } i$.

Q4: What is turnaround time?

A: The total time from when a process first arrives (becomes ready) until it completes execution. Formula: $TAT = \text{Burst Time} + \text{Waiting Time} = \text{Completion Time} - \text{Arrival Time}$.

Q5: What is the Convoy Effect in FCFS?

A: When a long CPU-bound process holds the CPU, many short processes accumulate in the Ready queue waiting. When the long process finally finishes, it also frees up I/O devices, leading to a burst of I/O activity — but then the short

processes finish quickly and I/O devices sit idle again. This alternating pattern of CPU and I/O underutilization is the convoy effect.

Q6: How do you calculate average waiting time?

A: Sum the waiting times of all processes and divide by the number of processes:

$$\text{Avg WT} = (\text{WT}[0] + \text{WT}[1] + \dots + \text{WT}[n-1]) / n.$$

Q7: Why does FCFS not suffer from starvation?

A: Every process in the Ready queue is guaranteed to eventually reach the head of the FIFO queue and be served. No process can be permanently delayed because there is no priority-based skipping.

Q8: In the FCFS simulation, why is `w_time[0] = 0`?

A: The first process to arrive (`P[1]`) waits for no other process — it immediately gets the CPU. Hence its waiting time is 0.

COMPILATION & EXECUTION QUICK REFERENCE

GCC Compilation Commands

Basic compilation (output: `a.out`)
`gcc filename.c`
`./a.out`

Compilation with named output
`gcc -o output_name filename.c`
`./output_name`

Compilation with command-line arguments
`gcc -o program filename.c`
`./program arg1 arg2 arg3`

Compile with warnings enabled (recommended)
`gcc -Wall -o program filename.c`

```
# Compile with debugging symbols (for gdb)
gcc -g -o program filename.c
```

Program-Specific Commands

Lab	File	Compile	Run
1.2i	Add_num.c	gcc -o Add_num Add_num.c	./Add_num 1 2 3 4
1.2i i	Minimum.c	gcc -o Minimum Minimum.c	./Minimum 1 2 3 2 1
1.2i ii	Avg.c	gcc -o Avg Avg.c	./Avg 1 2 3 2
1.3	env_var.c	gcc -o env_var env_var.c	./env_var
2	process_syscall s.c	gcc -o process_syscalls process_syscalls.c	./process_syscal ls
3	fcfs.c	gcc -o fcfs fcfs.c	./fcfs

Understanding argc for Each Lab 1.2 Program

Command: ./Add_num 1 2 3 4
argc = 5
argv[0] = "./Add_num" (program name)
argv[1] = "1"
argv[2] = "2"
argv[3] = "3"
argv[4] = "4"
nParams = argc - 1 = 4 (number of user-provided numbers)

Loop range for arithmetic: i = 1 to nParams (inclusive)

i=1: argv[1] = "1" → atoi → 1
i=2: argv[2] = "2" → atoi → 2
i=3: argv[3] = "3" → atoi → 3
i=4: argv[4] = "4" → atoi → 4

COMMON ERRORS & FIXES

Error	Cause	Fix
undefined reference to 'atoi'	Missing <code>#include <stdlib.h></code>	Add <code>#include <stdlib.h></code>
'INT_MAX' undeclared	Missing <code>#include <limits.h></code>	Add <code>#include <limits.h></code>
implicit declaration of function 'fork'	Missing <code>#include <unistd.h></code>	Add <code>#include <unistd.h></code>
implicit declaration of function 'wait'	Missing <code>#include <sys/wait.h></code>	Add <code>#include <sys/wait.h></code>
error: 'average' undeclared	Variable name mismatch in <code>Avg.c</code>	Rename <code>average</code> → <code>avg</code>
Segmentation fault in <code>Avg.c</code>	Integer division truncation	Cast: <code>(float)sum / nParams</code>
<code>./a.out: Permission denied</code>	File not executable	Run <code>chmod +x a.out</code>
Zombie processes appear in <code>ps</code>	<code>wait()</code> not called in parent	Add <code>wait(&status)</code> in parent block
<code>fork()</code> returns -1	Process limit reached or insufficient memory	Check <code>ulimit -u</code> ; reduce process count
FCFS gives wrong avg (decimal truncated)	Integer division	Cast: <code>(float)avg_wtime / n</code> for float result

Key Formulae Cheat Sheet

— CLI ARGUMENTS:

argc = total arguments (including program name)
nParams = argc - 1 (user-provided count)
argv[0] = program name
argv[1..] = user arguments (as strings, use atoi() to convert)

— PROCESS SYSTEM CALLS:

fork() → 0 (child) | child_PID (parent) | -1 (error)
getpid() → PID of calling process
wait(&s) → blocks parent until a child terminates
exit(0) → terminates process successfully

— FCFS SCHEDULING:

WT[0] = 0
WT[i] = WT[i-1] + BT[i-1] (O(n) formula)
TAT[i] = BT[i] + WT[i]
Avg WT = $\sum(WT[i]) / n$
Avg TAT = $\sum(TAT[i]) / n$

—
End of OS Lab Master Notes — Labs 1 to 3
RV University | CS1103 Lab | Semester II